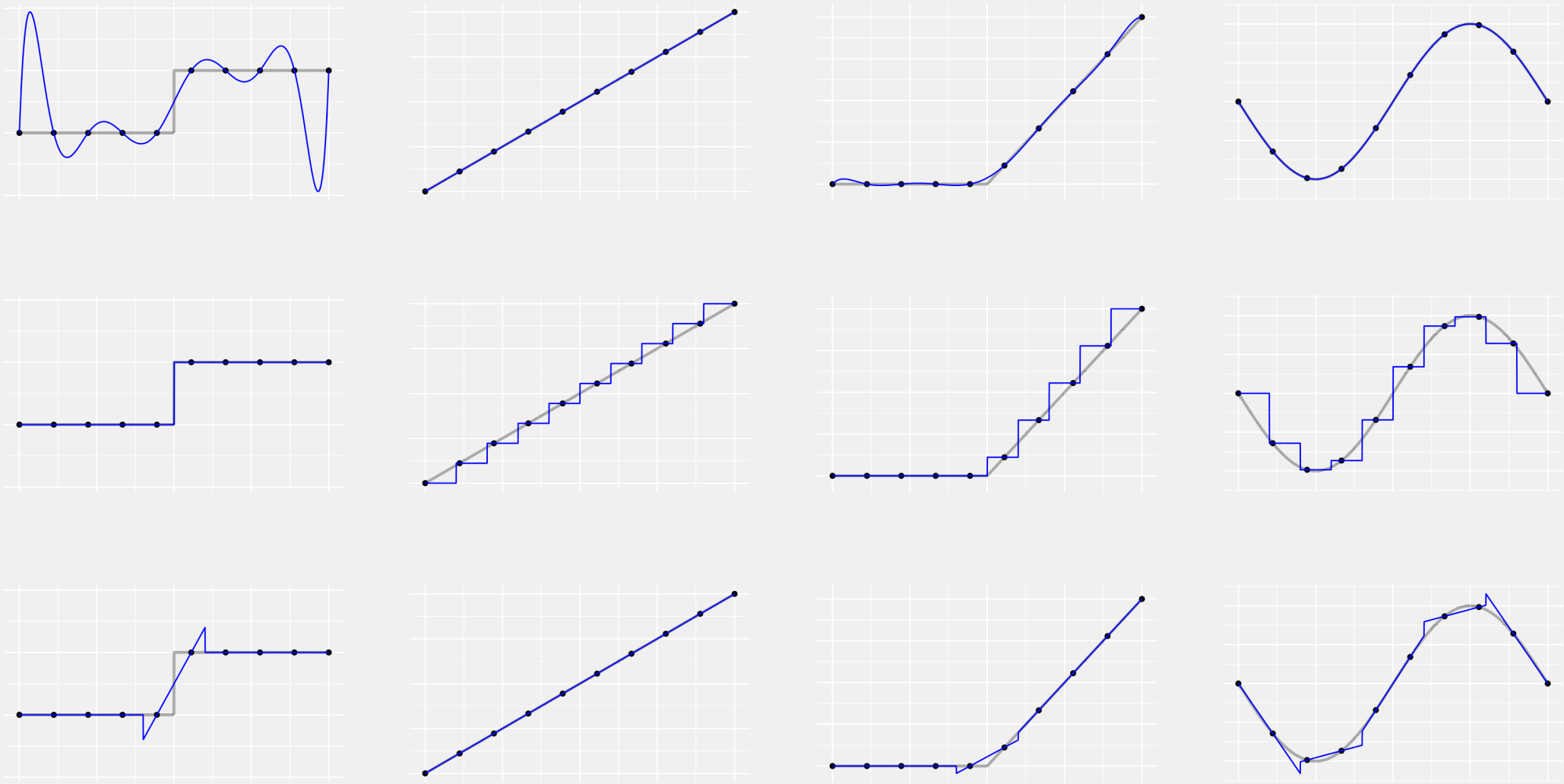
Fitting Curves Better

QTM 220 Week 3

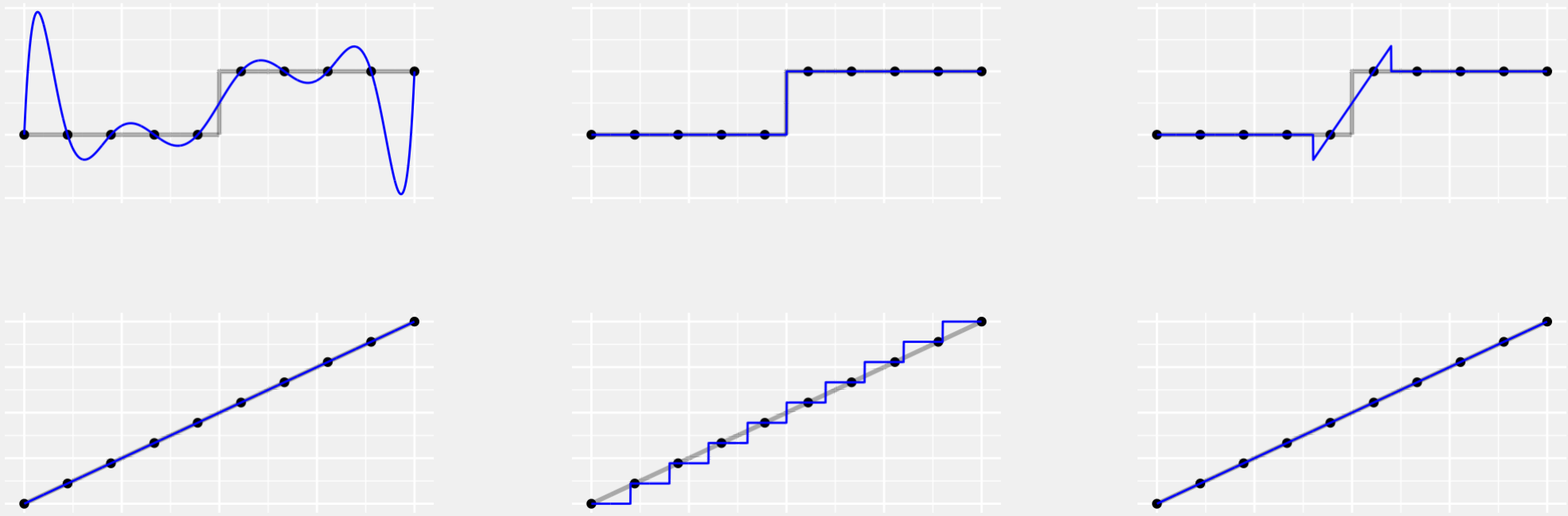
Lessons from our last lab

There are good and bad ways to fit the data.



We fit 10 observations with 10-dimensional models.

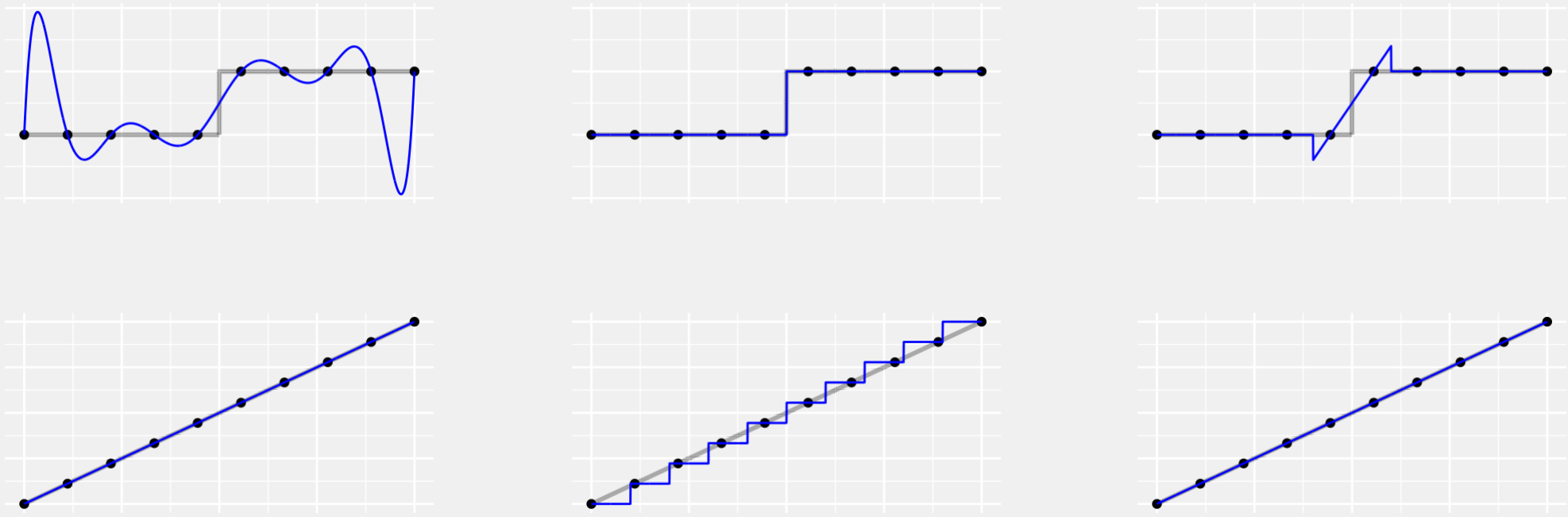
This let us fit the observations we had perfectly.



What we got between them depended on the model.

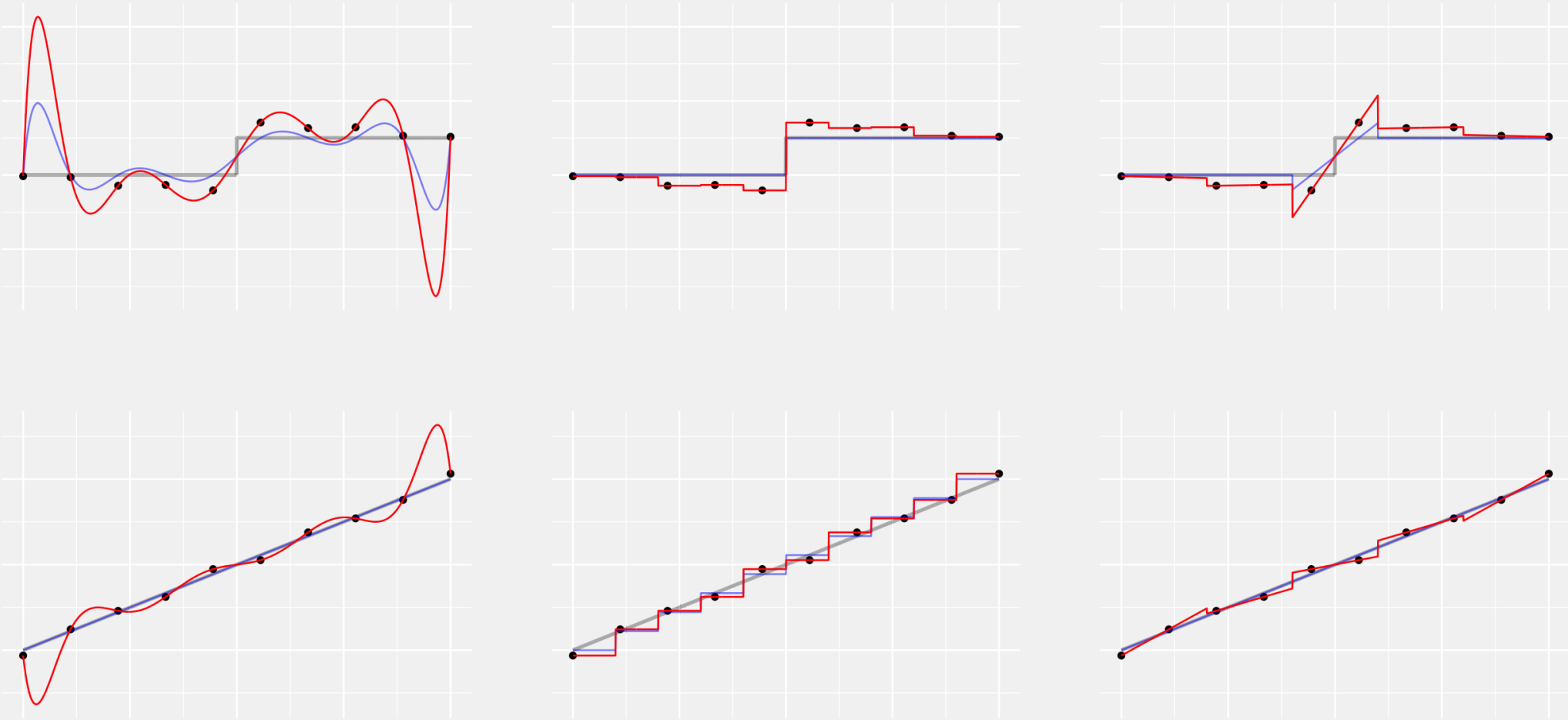
We fit 10 observations with 10-dimensional models.

This let us fit the observations we had perfectly.



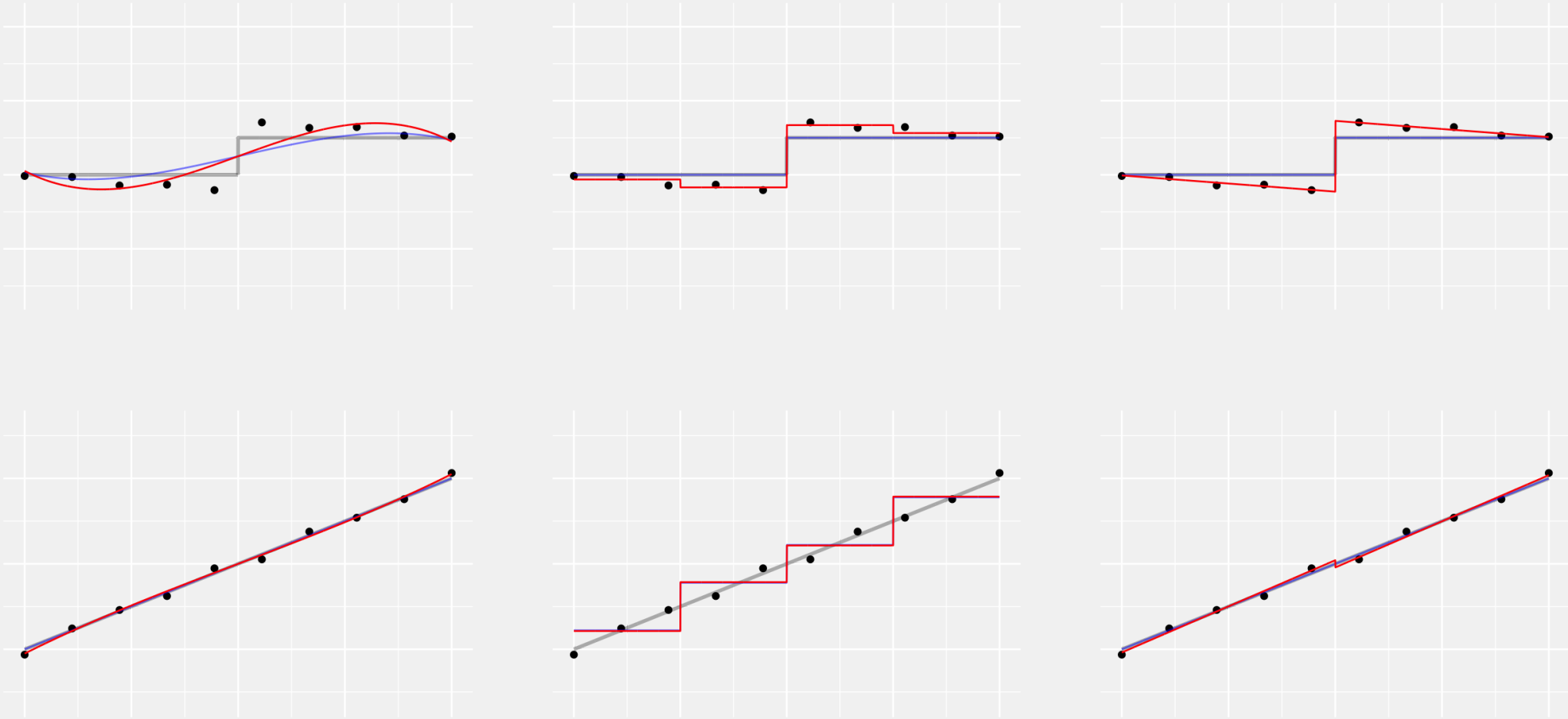
- Wiggling can be very bad in places.
- Staircasing causes smaller problems.
- Kinking is somewhere in between.

This matters more if you observe the curve with error



Even if the errors are pretty small, like these compression errors.

But less so when you use smaller models

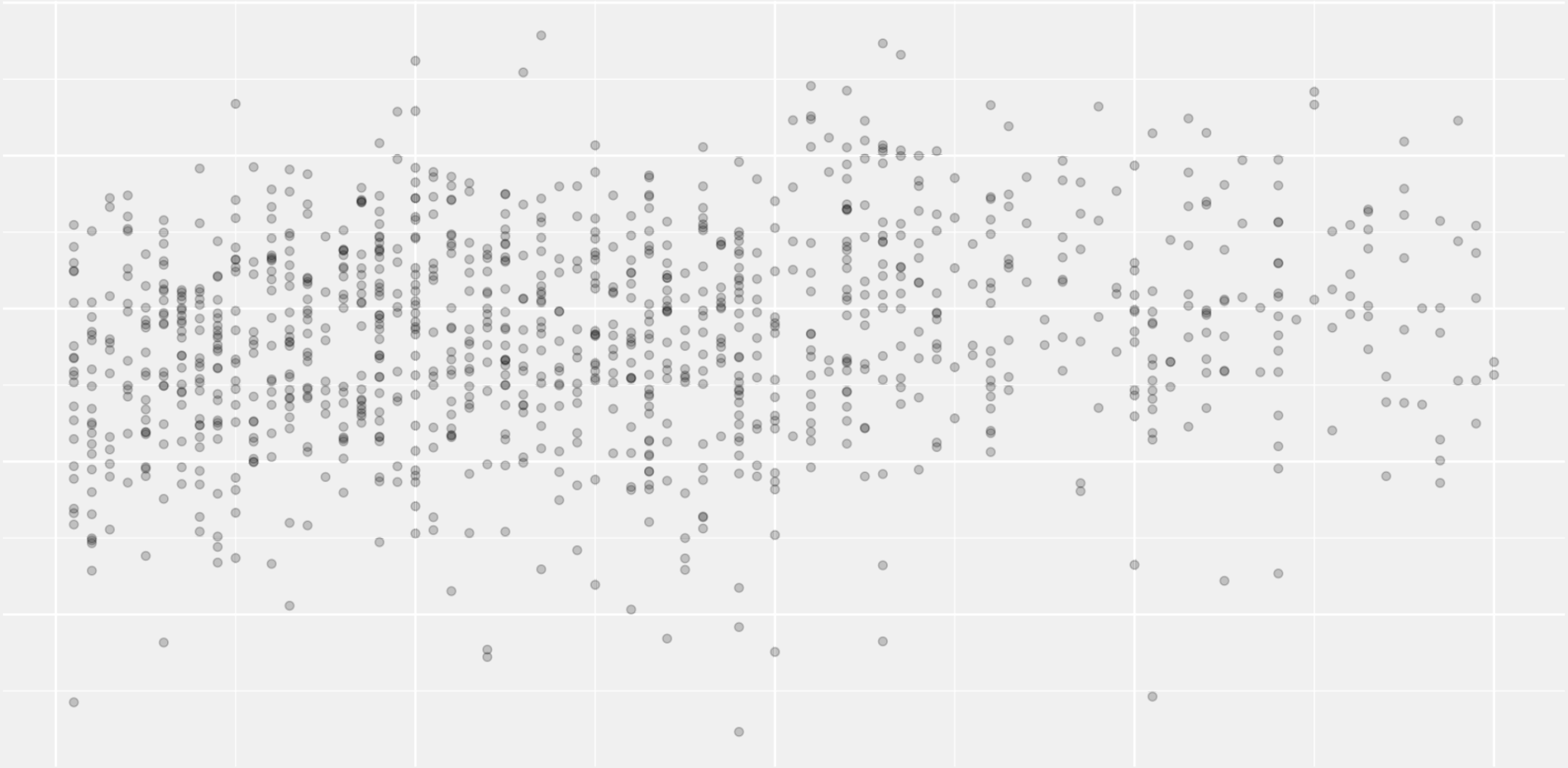


These ones are four parameter models.

That’s still pretty big

- One rule of thumb is that you want ten observations per parameter.
  - That only justifies fitting a constant to the ten we have.
  - But we’ve been able to do very well with these larger models.
  - Especially in the zero-errors case.
- That rule of thumb assumes errors are of **typical size**.
  - Our errors here have been zero or very small.
  - That’s what allows us to get away with relatively large models.
  - And that’s what a lot of signal processing applications look like.

That’s not what all applications look like



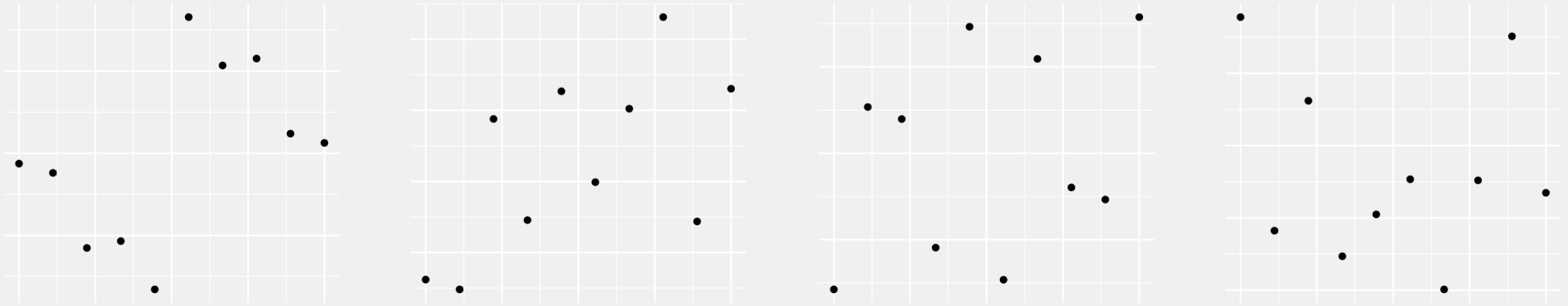
Test scores vs. enrollment in fake Israeli schools

Enrollment has some predictive power, but there’s a ton of variation.

To get closer to these applications, let's scale up our errors

- We can't really do it with the compression errors we've been using.
- But we can do it with random errors.
- And often, random errors will give you the same behavior.
- You can think of random errors as a **model** for your non-random ones.

Here's our compression errors and random ones with matched variance.

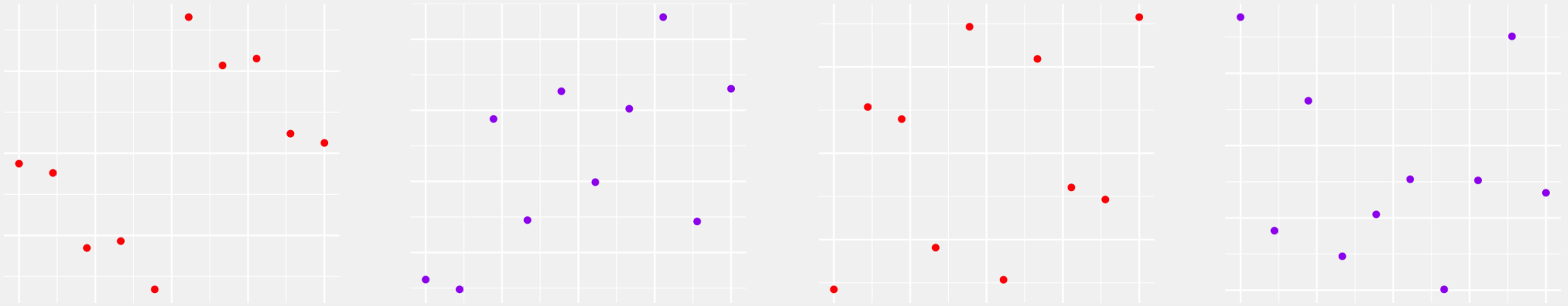


Take a look. Guess which is which.

To get closer to these applications, let's scale up our errors

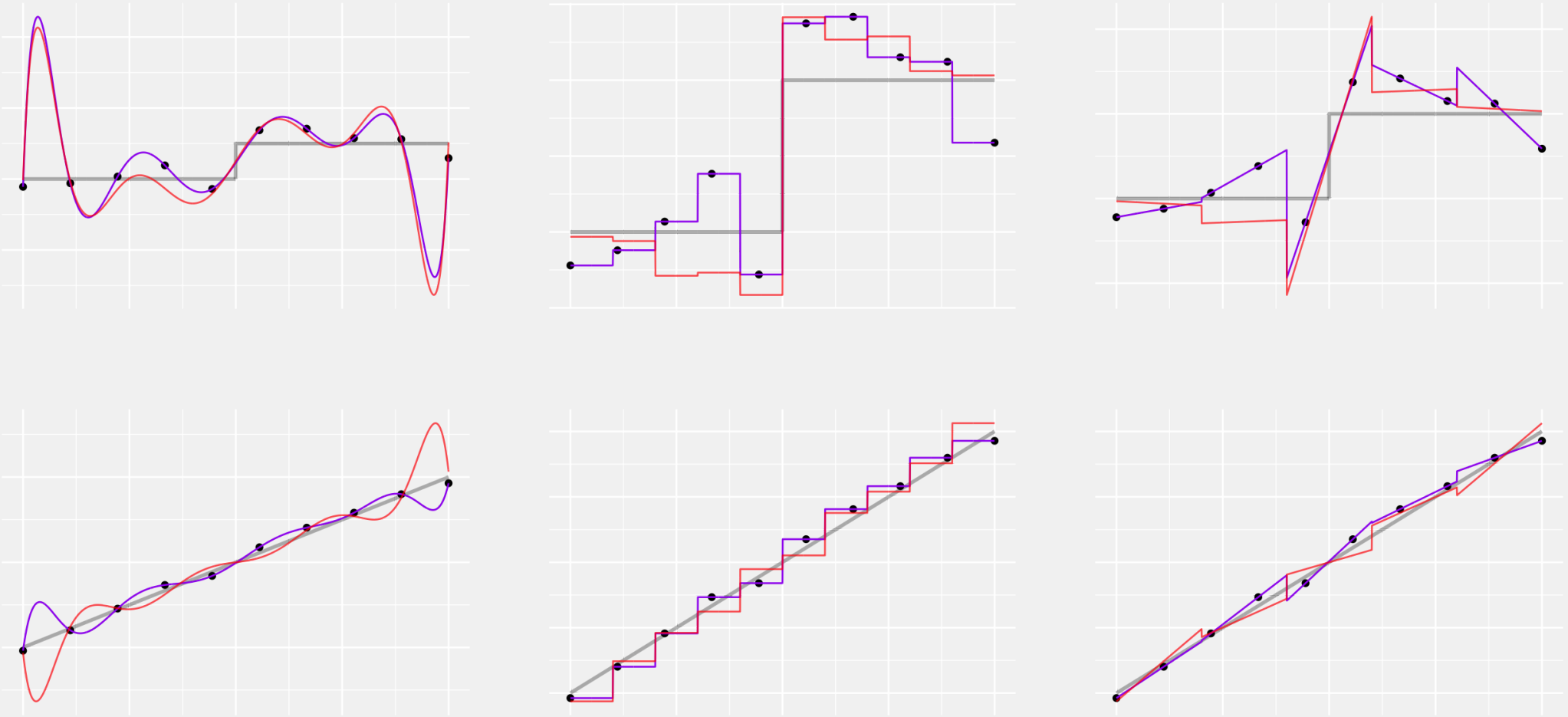
- We can't really do it with the compression errors we've been using.
- But we can do it with random errors.
- And often, random errors will give you the same behavior.
- You can think of random errors as a **model** for your non-random ones.

Here's our compression errors and random ones with matched variance.



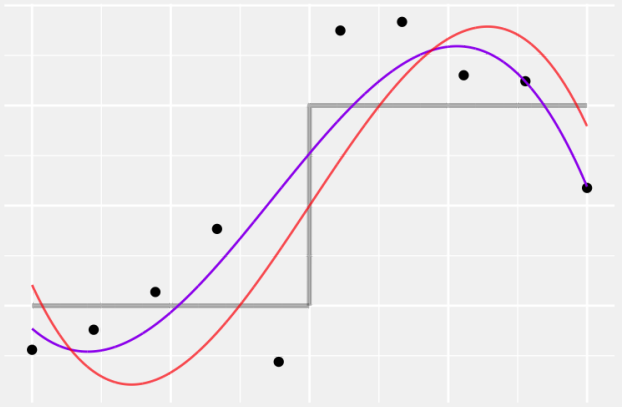
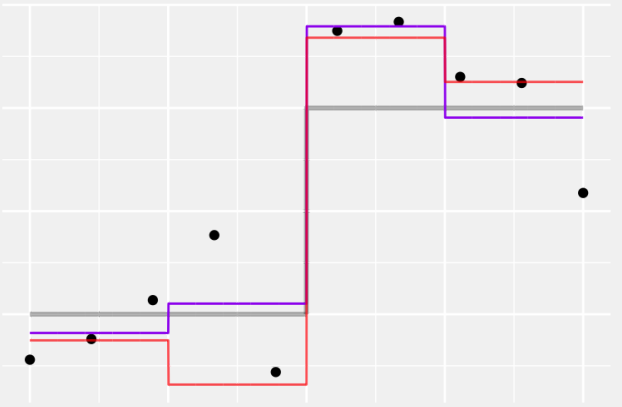
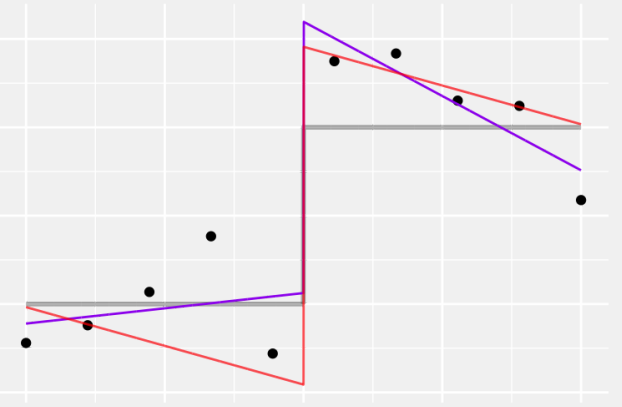
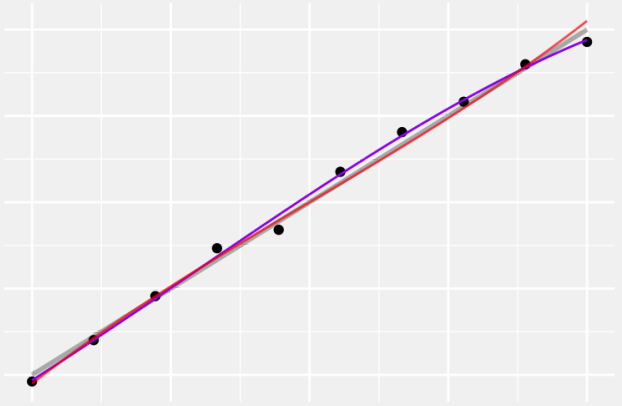
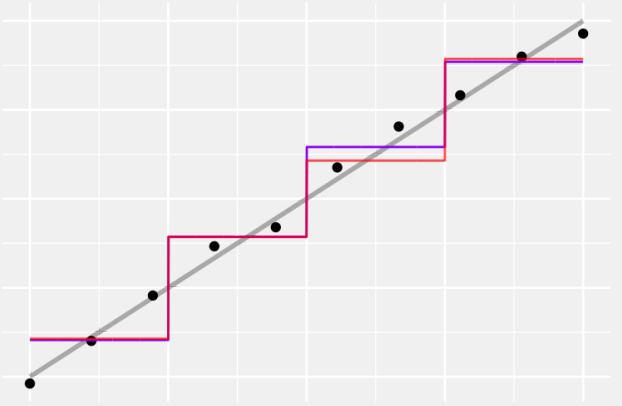
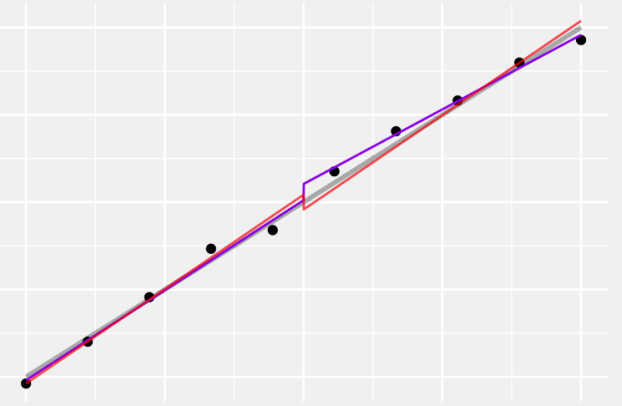
The red ones are compression errors. Let's fit models with the purple.

Random errors vs. Compression Errors



The behavior of our interpolating fits is qualitiatively similar.

Random errors vs. Compression Errors

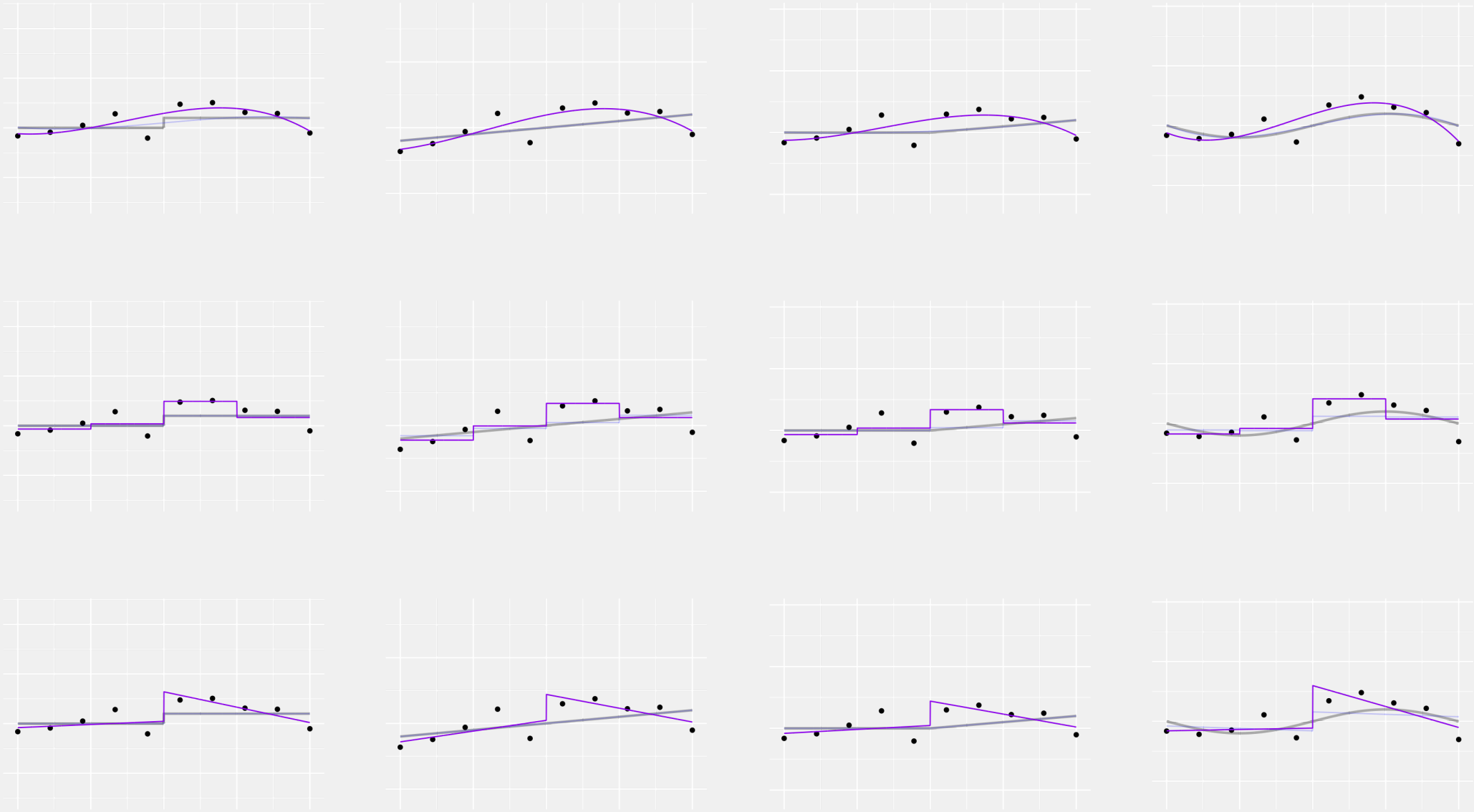







It’s similar when we fit smaller models, too.

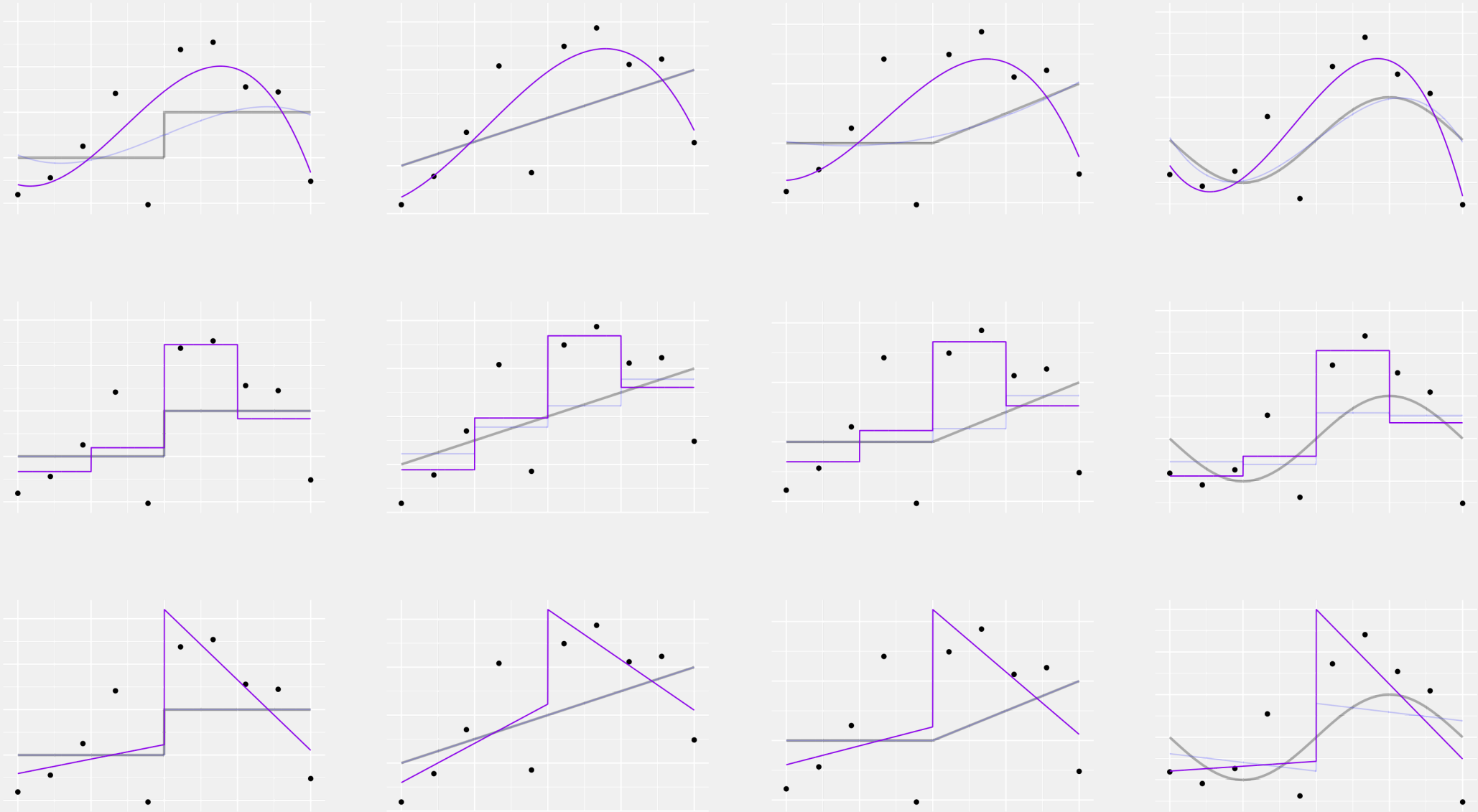
What Happens at Typical Error Levels

These models are too big.

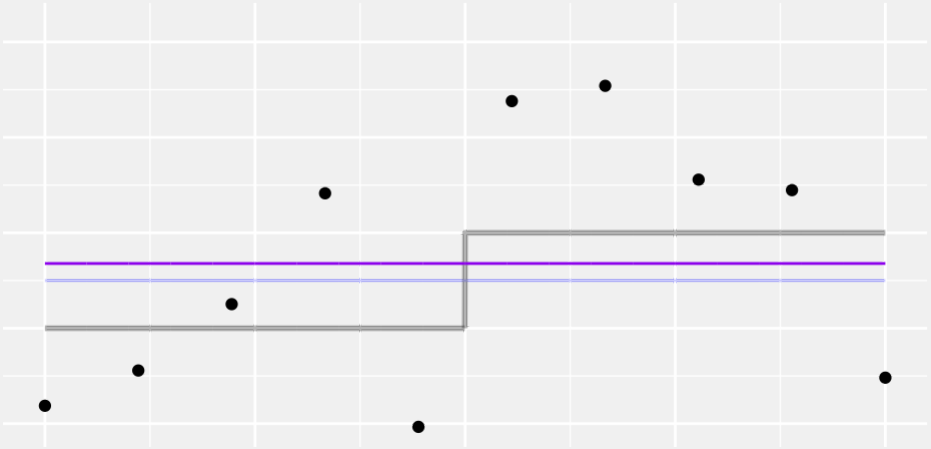
These models are too big.

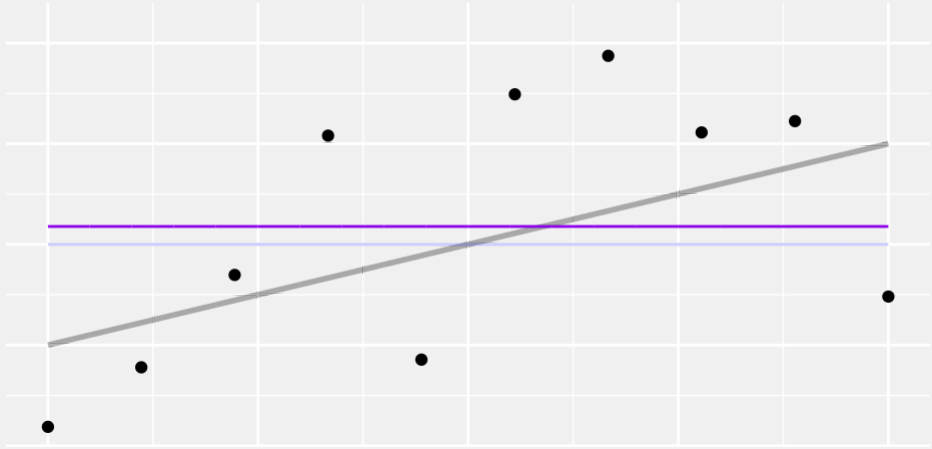


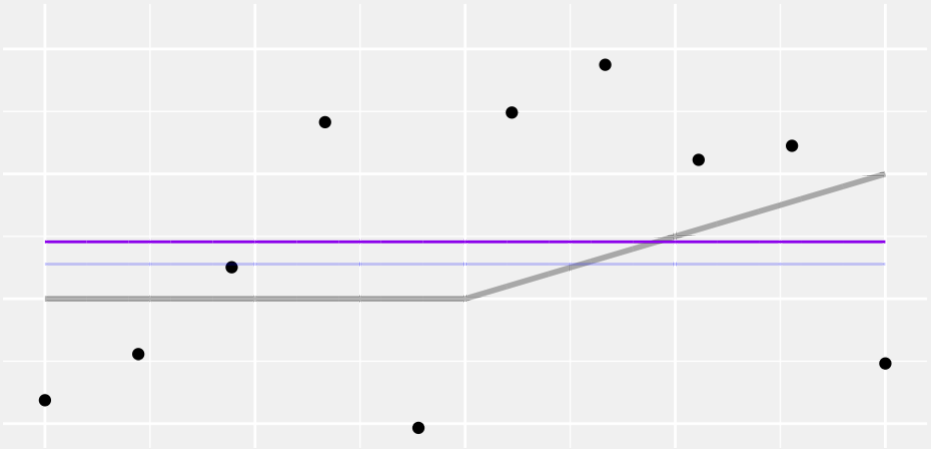
These models are too big.

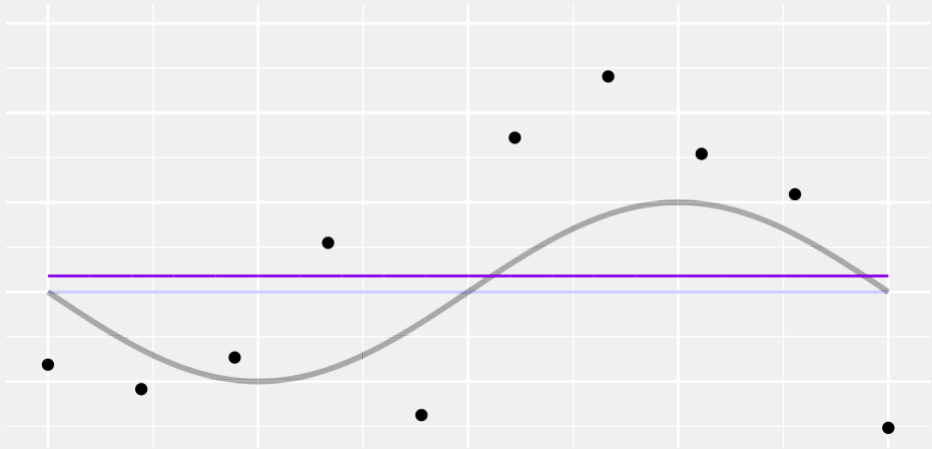


We could fit a constant.

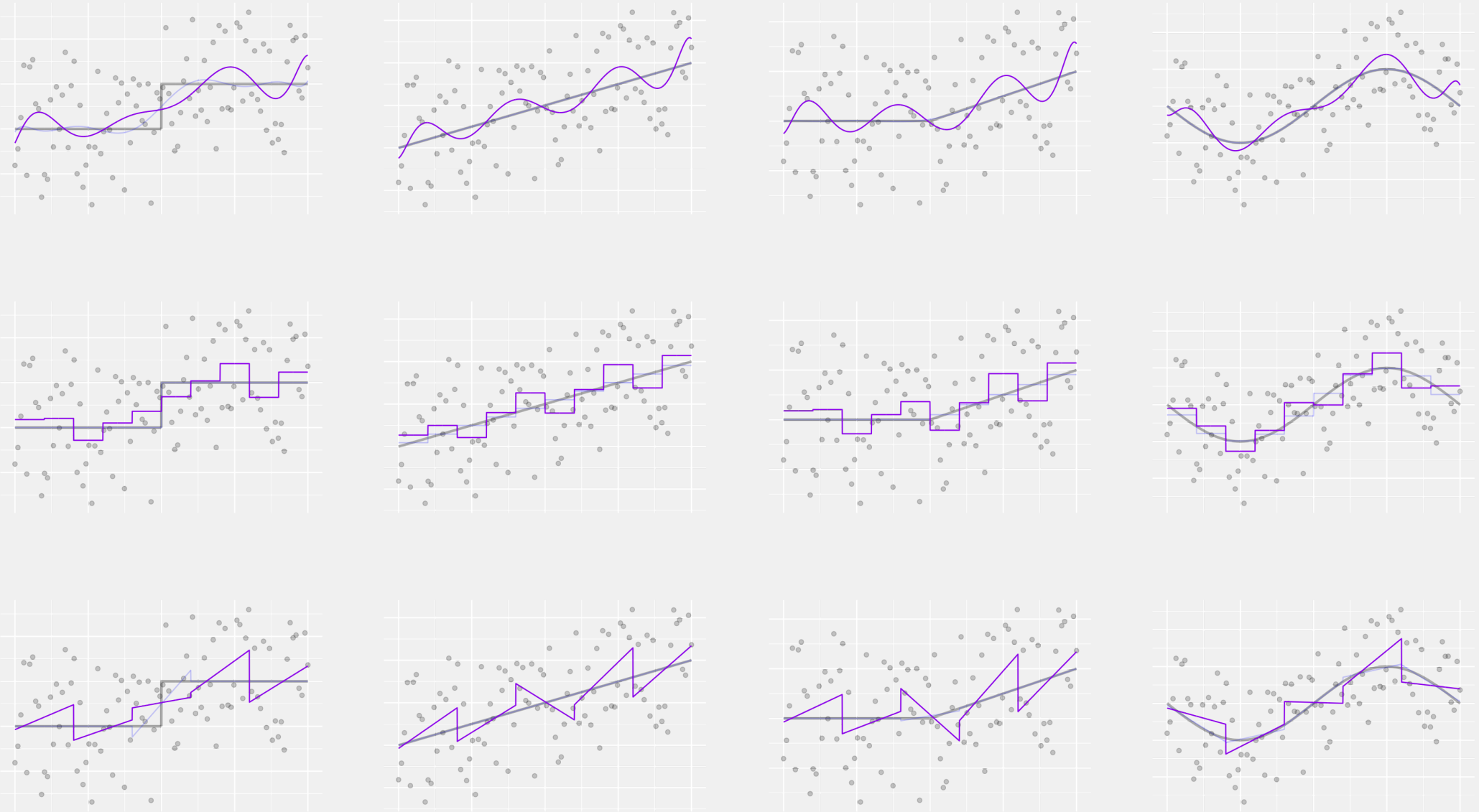




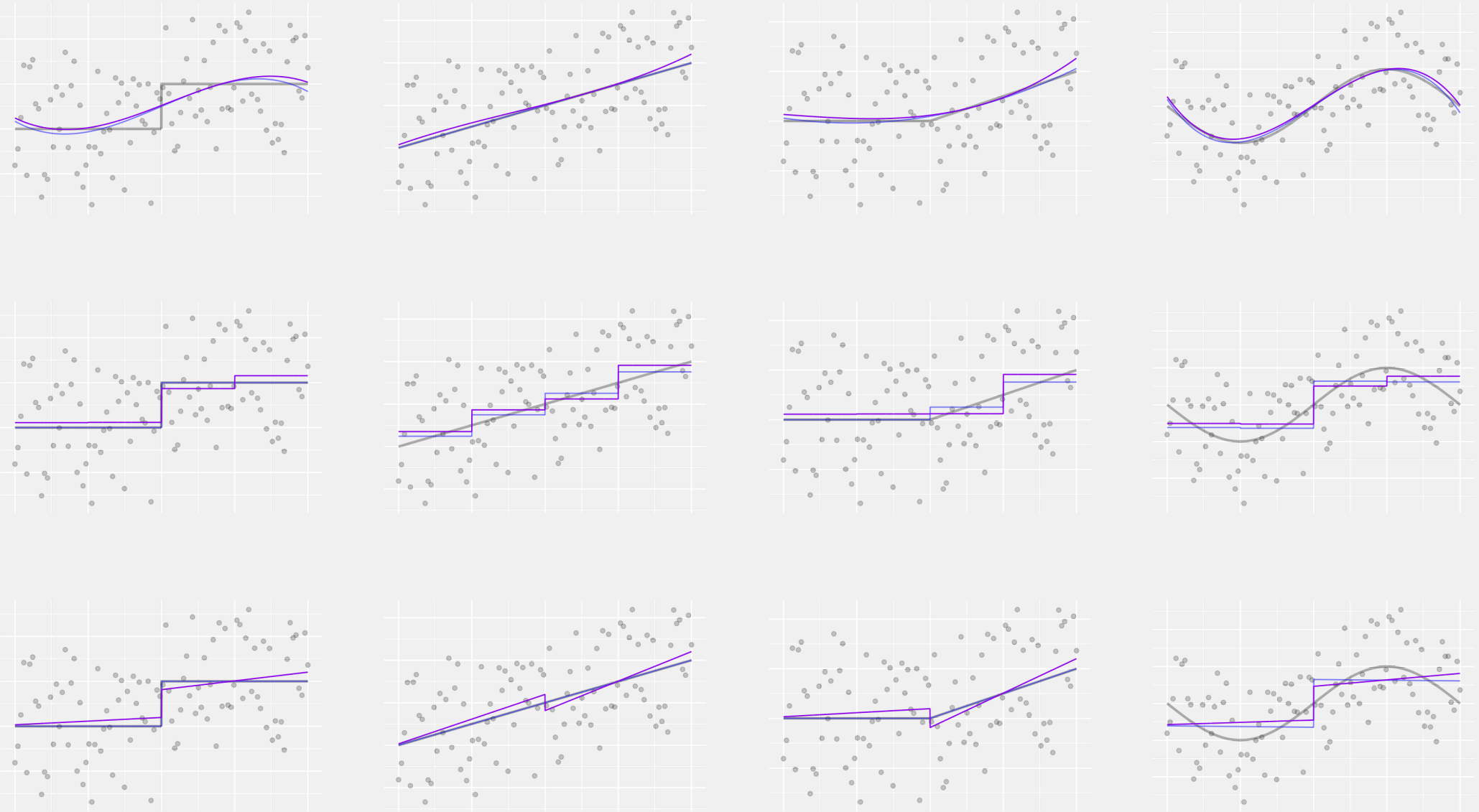




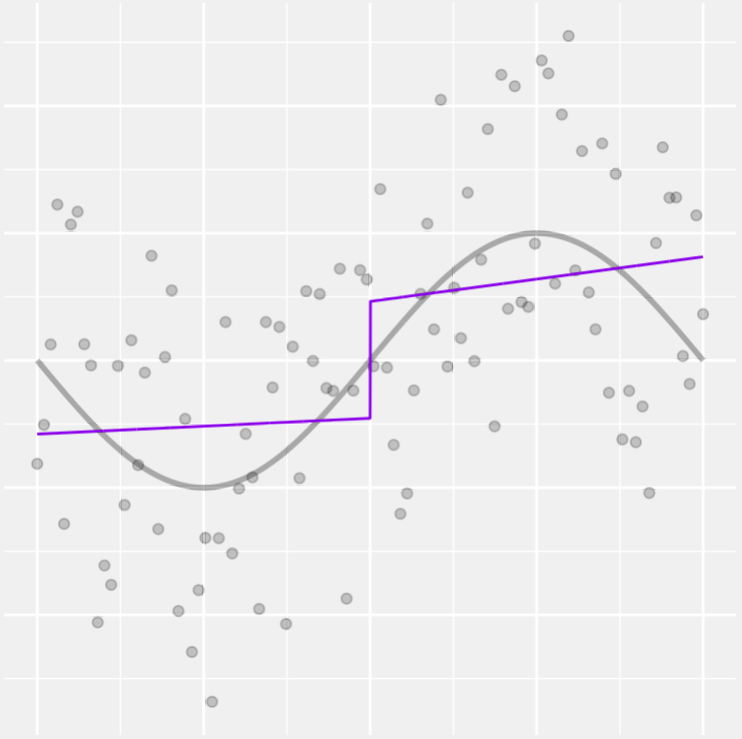
Usually we have more.

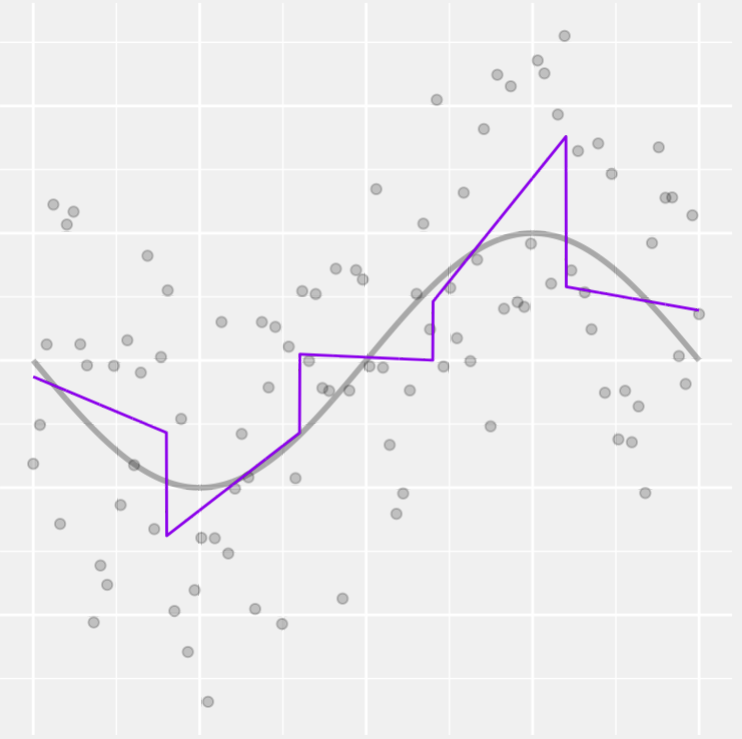


Usually we have more.



But we're left with an uncomfortable choice





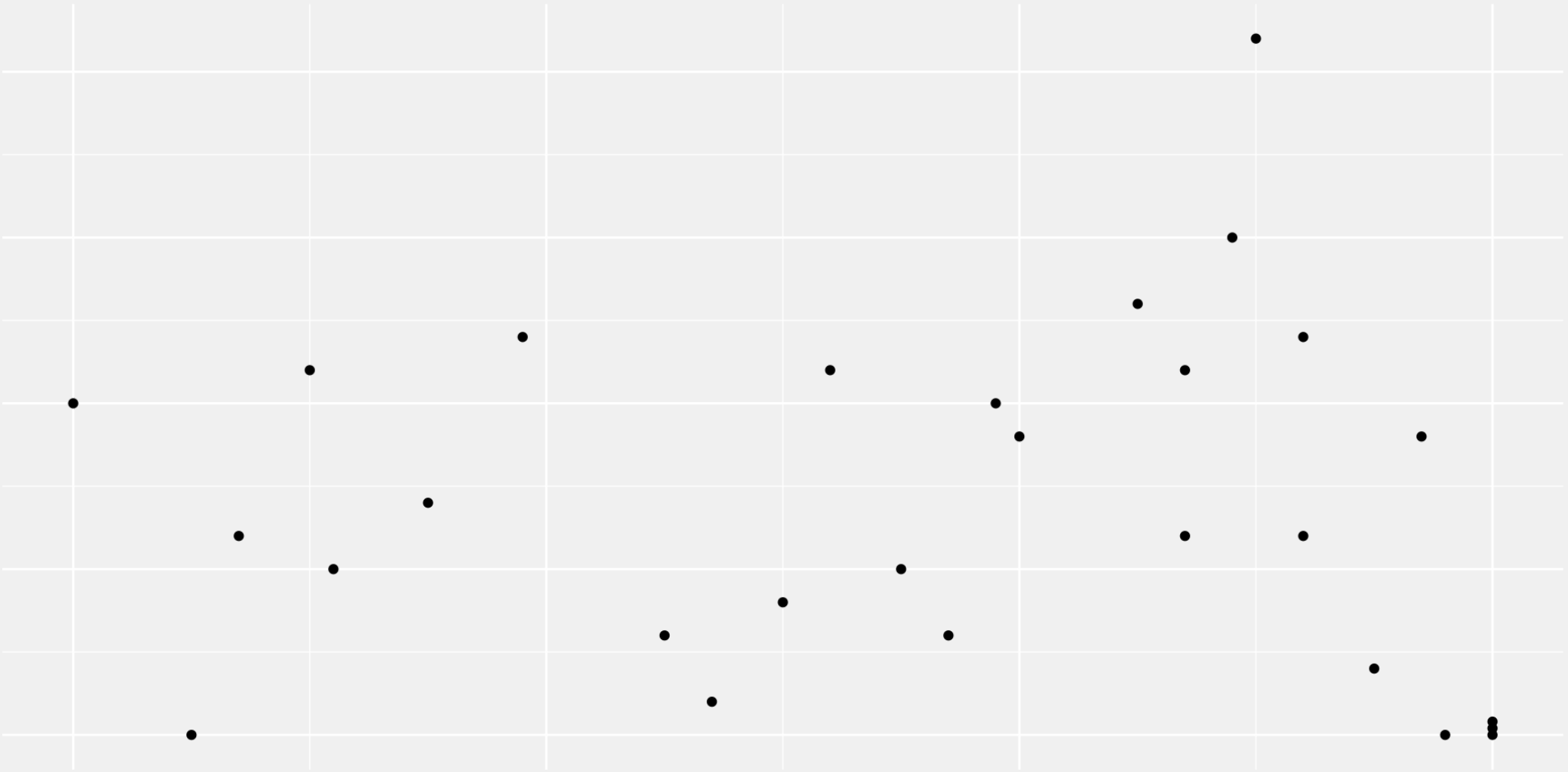
- The smaller models can't always capture the trend.
- The bigger models exhibit some weird artifacts.

There’s got to be something better than this

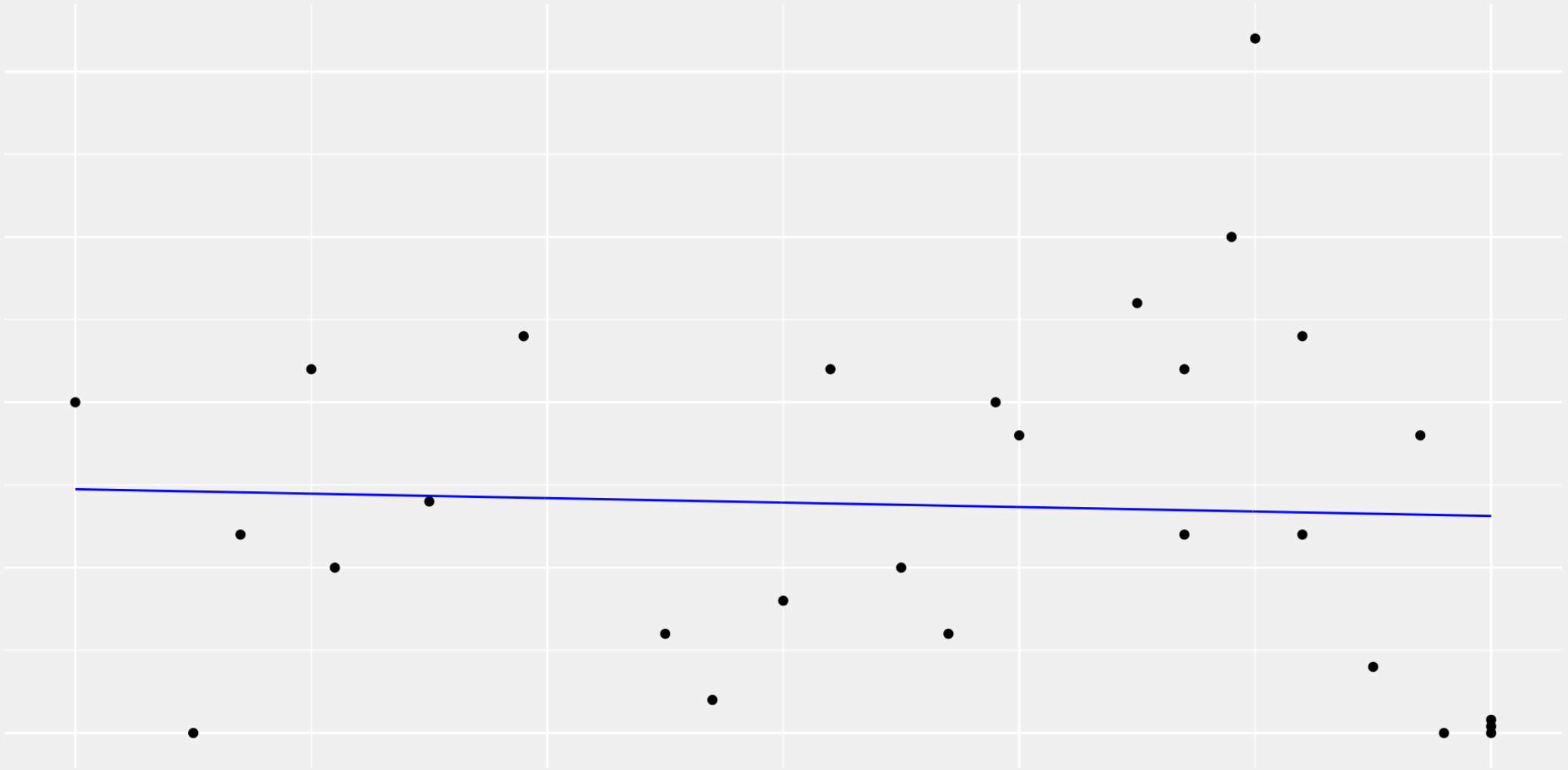
- There is an inevitable trade-off between the flexibility to fit the stuff we want and tendency to fit the stuff we don’t.
- But the knobs we have to fiddle with may not be the ones we want.
- If we’re talking about piecewise-polynomial fits, we’ve got two knobs.
  - The degree of each polynomial.
  - The number of breaks.
- These don’t complement each other very well.

Let’s fiddle with them a little and try to fit some data

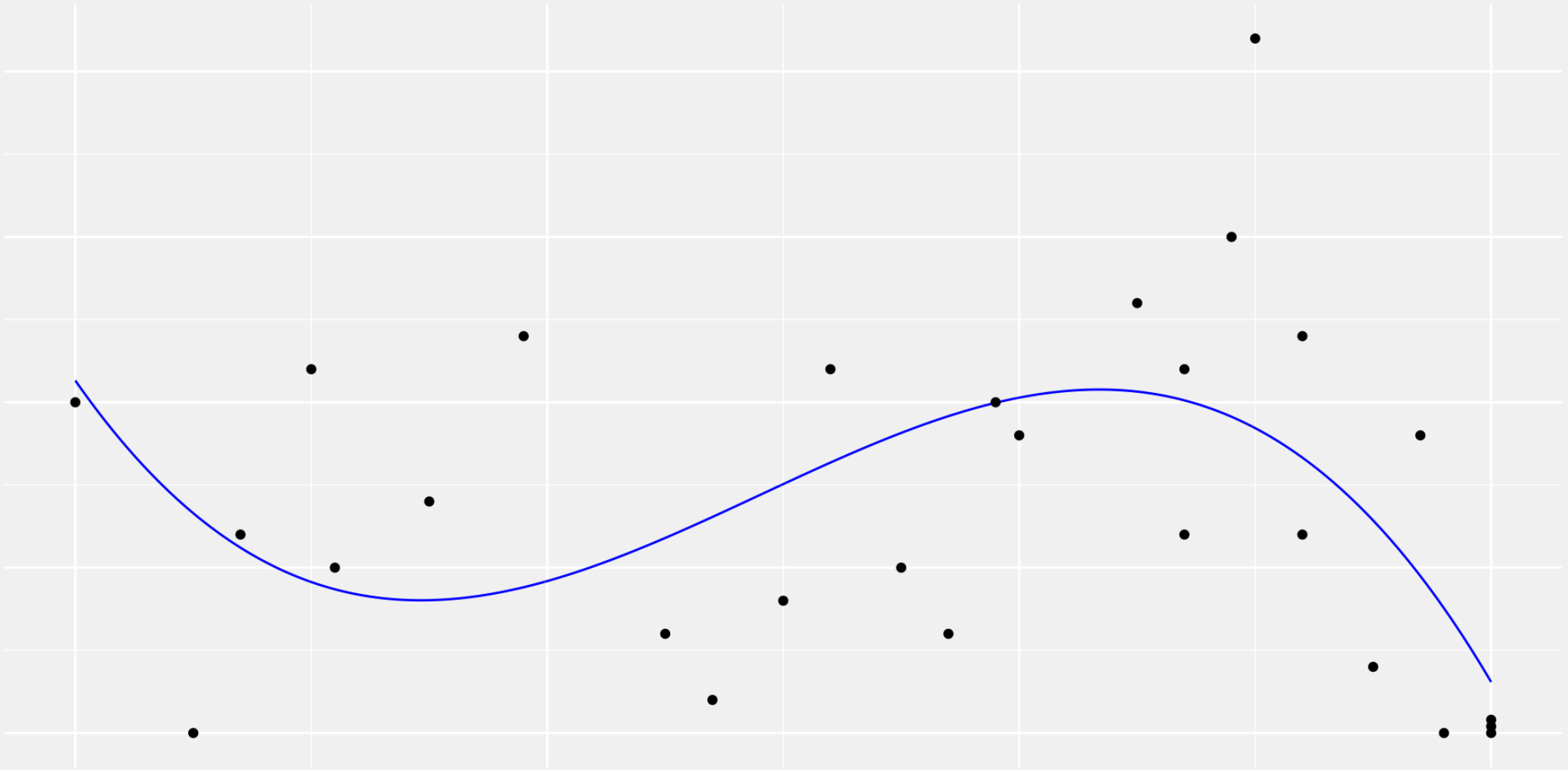
The Data



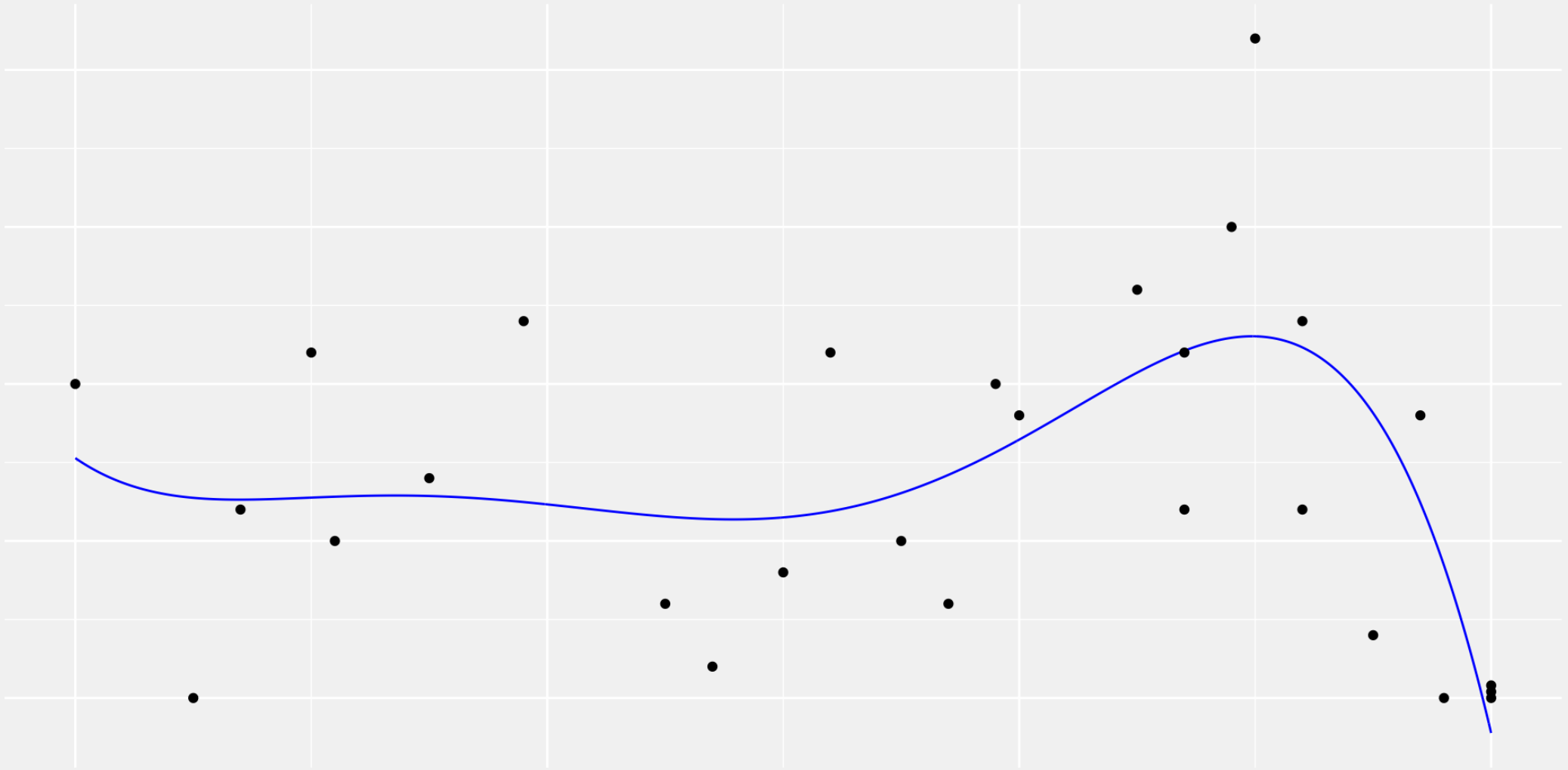
Linear fit



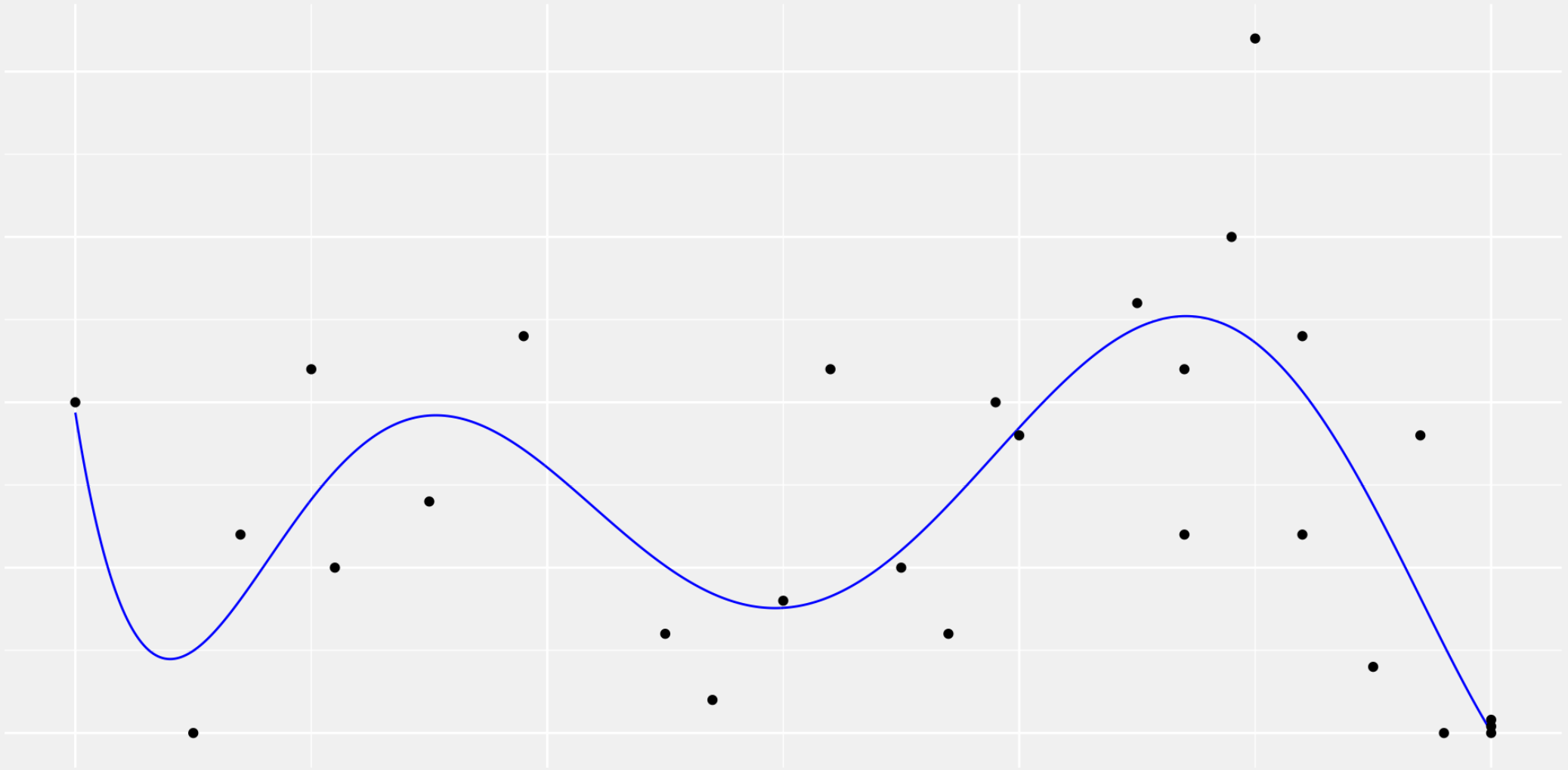
Cubic polynomial fit



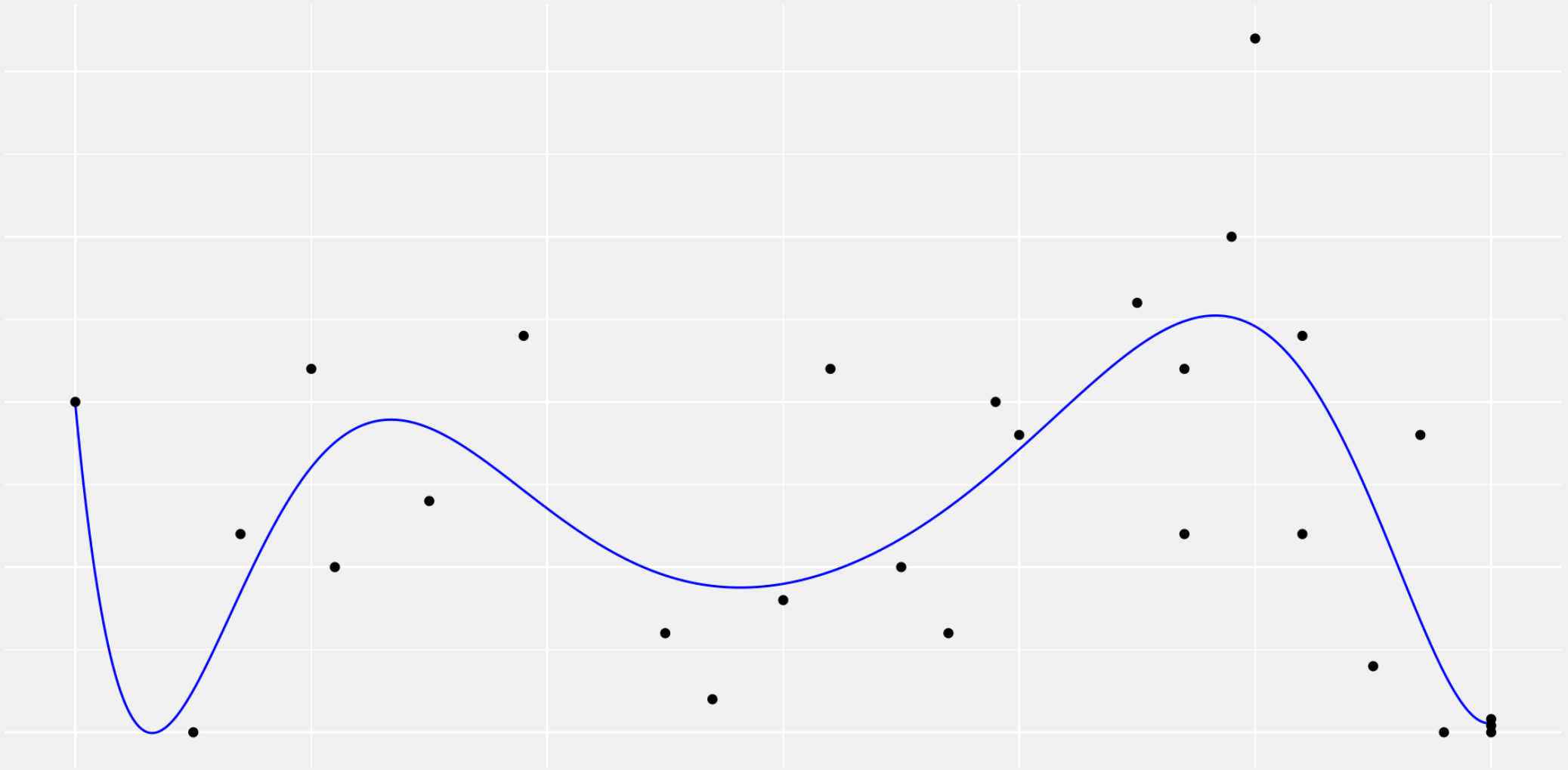
5th-order polynomial fit



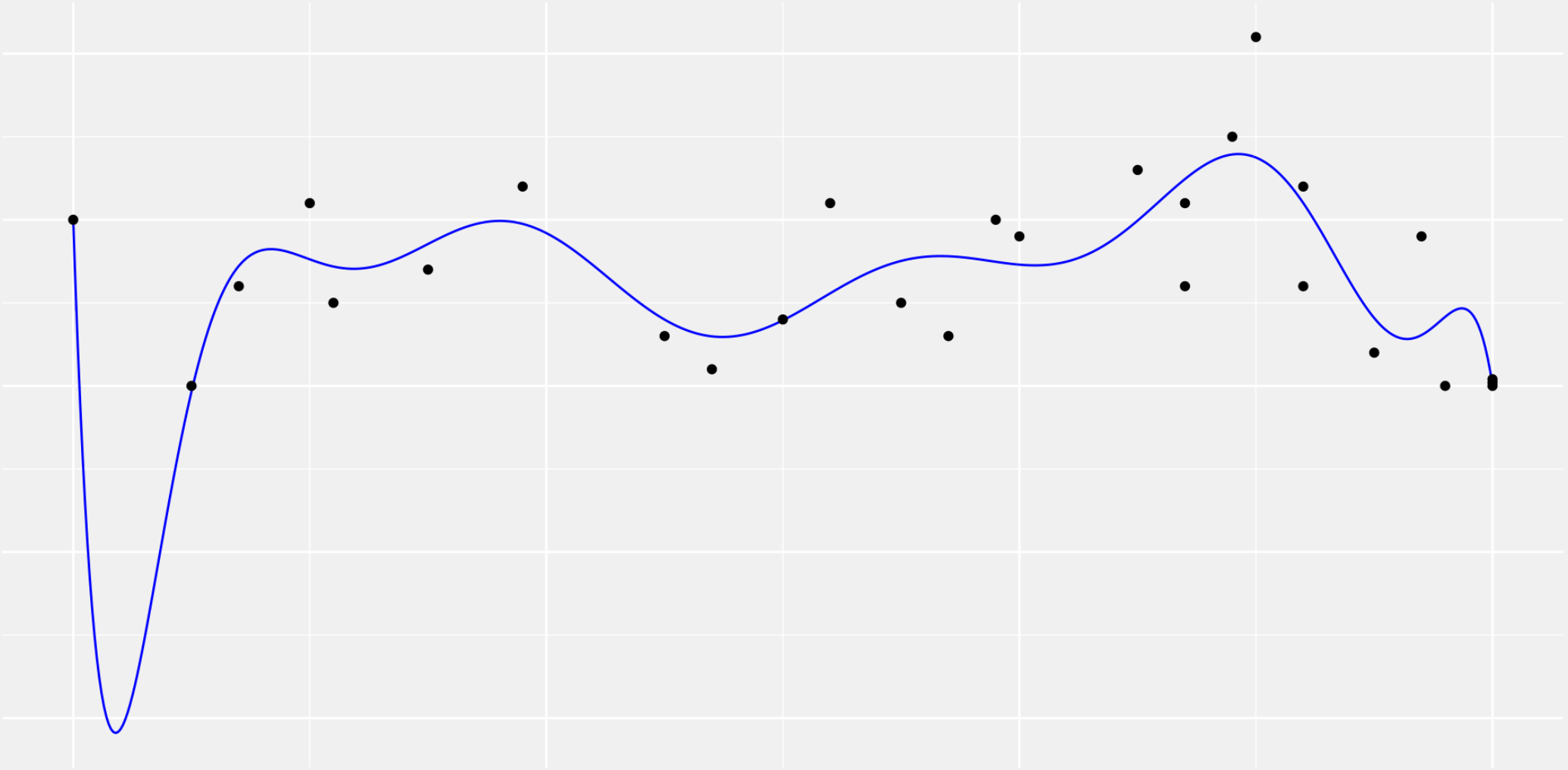
7th-order polynomial fit



9th-order polynomial fit



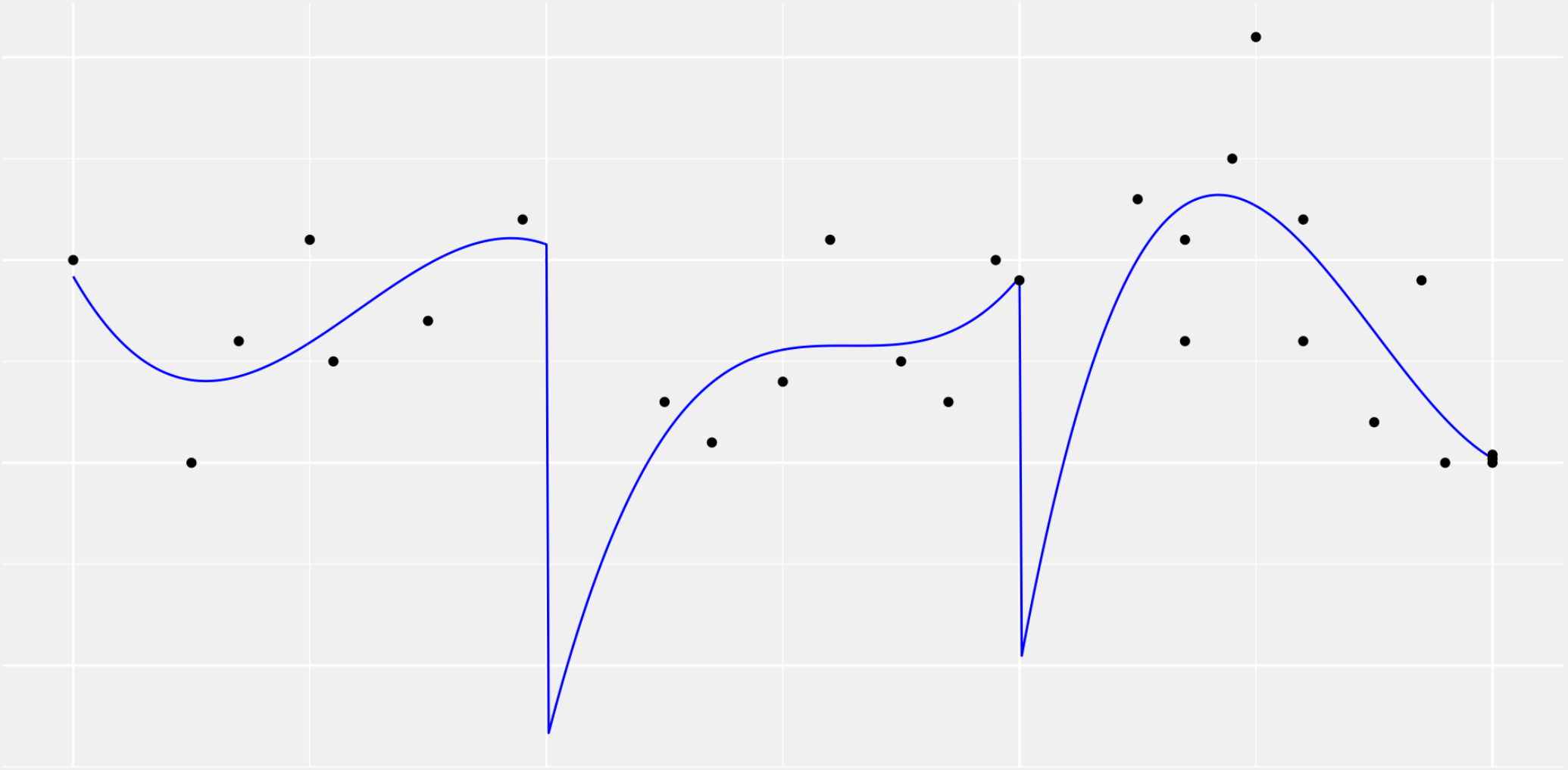
11th-order polynomial fit



This just doesn't work

- Low-order polynomials (e.g. lines, cubics) can't fit the data.
- High-order polynomials exhibit weird **non-local** behavior.
- To fit better where there's more data,
  they'll do crazy things where there isn't much.
- You might hope that a piecewise polynomial fit would be better.

A piecewise-cubic fit



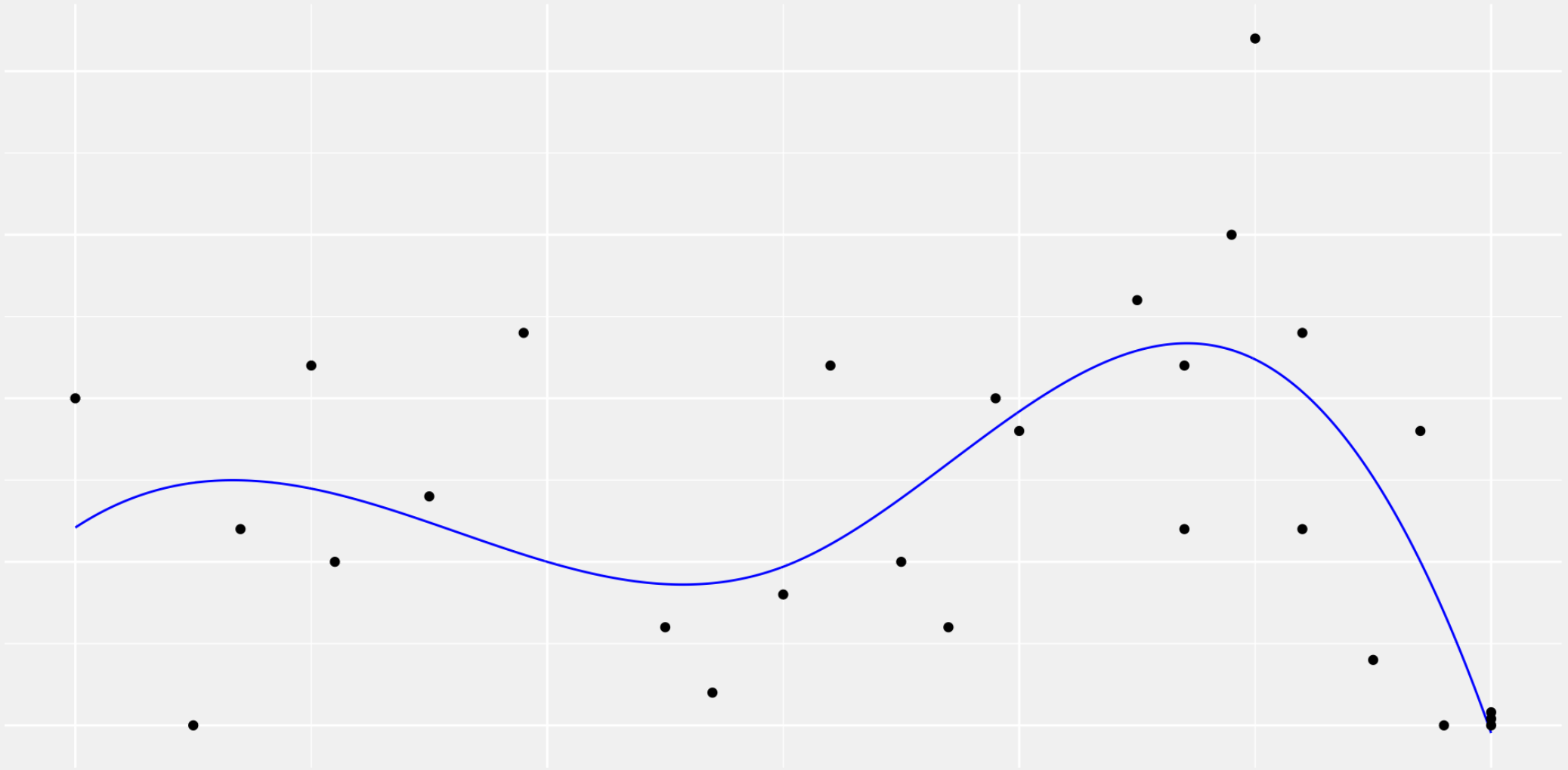
This may be even worse.

- Polynomials can do weird things at the boundaries of the data.
- Now we've put boundaries in the middle of the data, too.
- These problems are like the ones from lab, but worse.
  - In lab, our observations were evenly spaced. That's as good as it gets.
  - Here, they're not. That's usually the case.

Fixing this problem is easy.

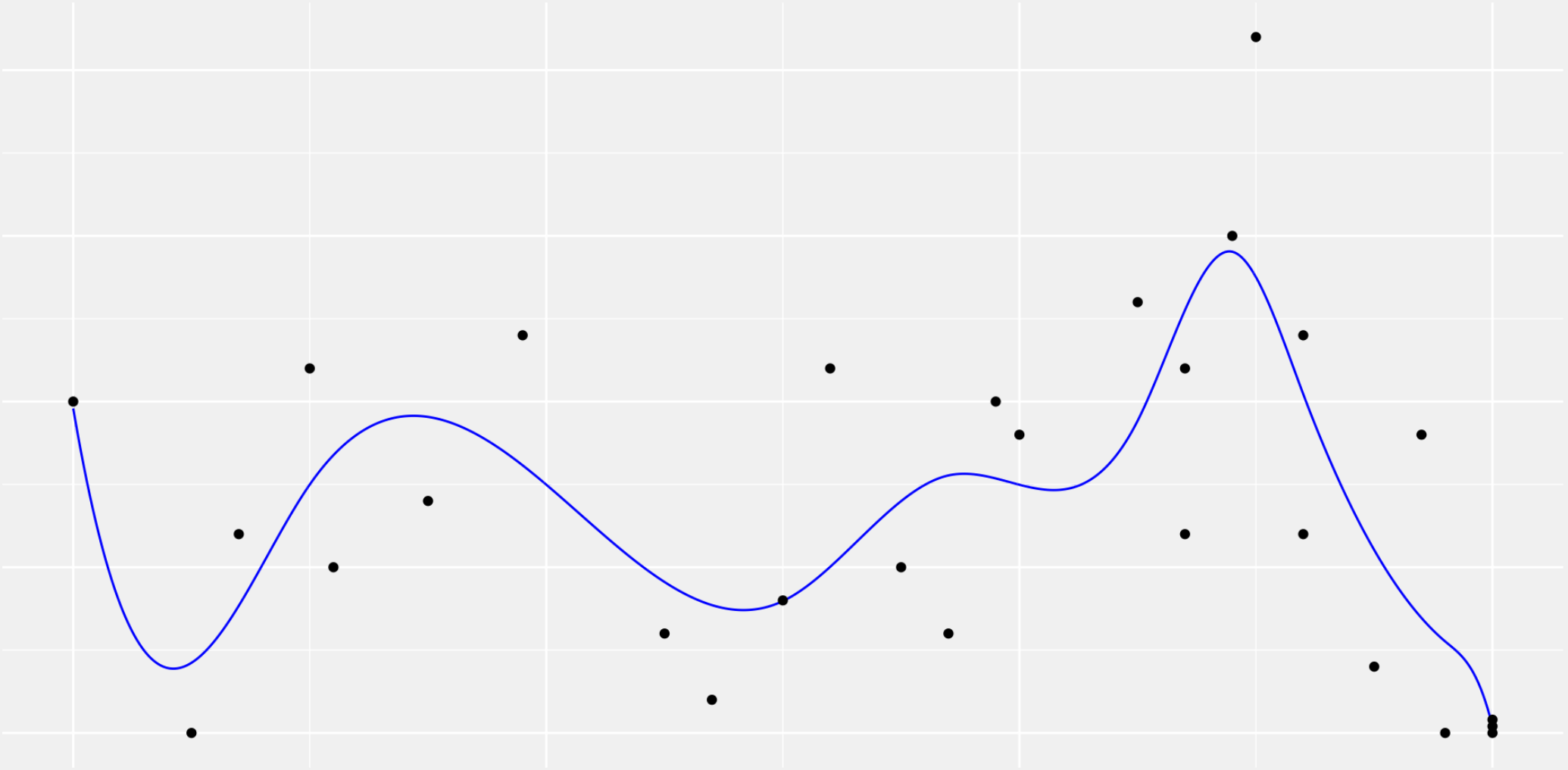
- Instead of using all piecewise-polynomial functions, use only ones that:
  - Are continuous, i.e., with polynomials that match at the breaks.
  - Are differentiable, i.e., with matched derivatives too.
  - Maybe twice differentiable, i.e. with matched second derivative too.
- These functions are called **splines**.
- The breaks, where we stitch our polynomials together, are called **knots**.
- Usually people use cubic polynomials with
  - Two matched derivatives
  - Enough knots to give the flexibility they want.

These work pretty nicely



A cubic spline with two knots.

Even with a lot of parameters

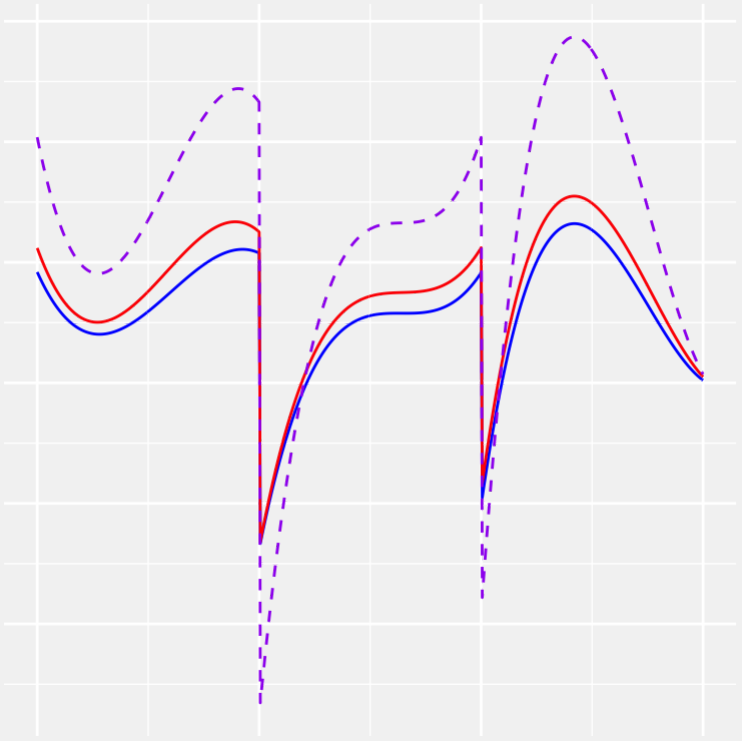


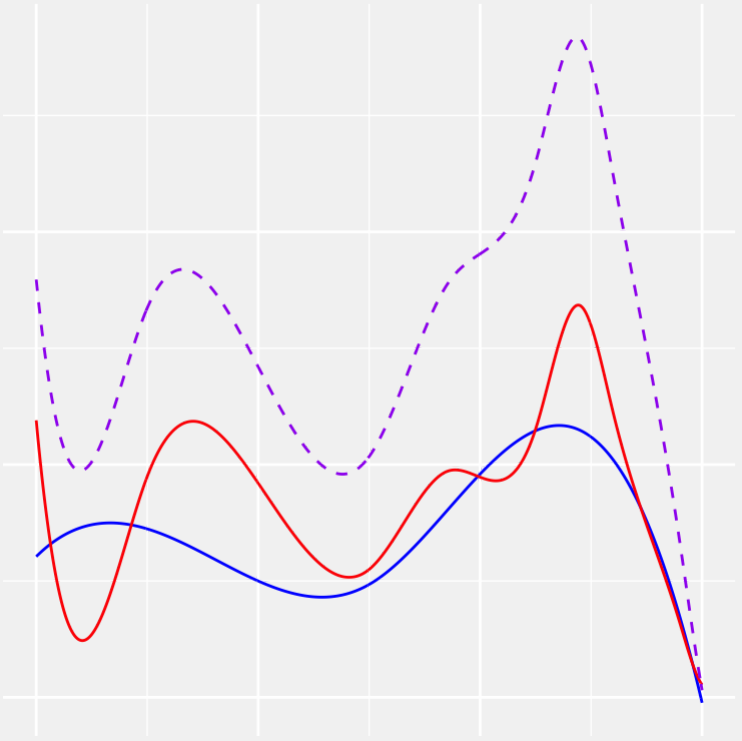
A cubic spline with 8 knots.

Spline Models

Splines are linear models.

- If you add two piecewise-polynomial curves, you get another.
- If you add add two differentiable curves, you get another.
- So if you add two differentiable piecewise-polynomial curves, ...





A Basis for Splines with Knots at Zero

For a cubic spline with one knot at zero, we can work out a general formula.

$$m_\beta(x) = \left(\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3\right) + \left(\beta_{0,+} + \beta_{1,+} x + \beta_{2,+} x^2 + \beta_{3,+} x^3\right) 1(x \geq 0).$$

with these continuity and differentiability constraints:

A Basis for Splines with Knots at Zero

For a cubic spline with one knot at zero, we can work out a general formula.

$$m_\beta(x) = \left(\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3\right) + \left(\beta_{0,+} + \beta_{1,+} x + \beta_{2,+} x^2 + \beta_{3,+} x^3\right) 1(x \geq 0).$$

with these continuity and differentiability constraints:

$$\beta_0 = \beta_0 + \beta_{0,+} \qquad \text{continuous at zero}$$
$$\beta_1 = \beta_1 + \beta_{1,+} \qquad \text{differentiable at zero}$$
$$\beta_2 = \beta_2 + \beta_{2,+} \qquad \text{twice-differentiable at zero}$$

which gives us the simplified form

A Basis for Splines

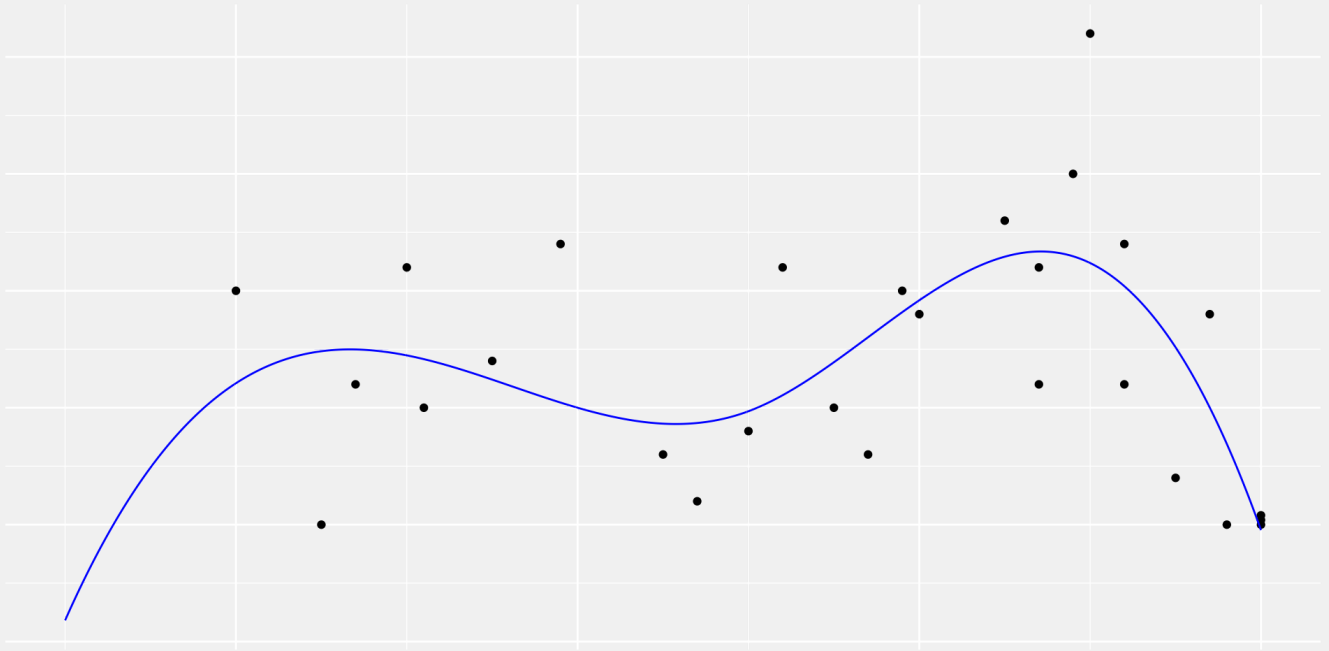
For a cubic spline with knots at $x_1 \ \ldots \ x_k$, there's a similar formula.

$$m_\beta(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \beta_{3,k}(x - x_k)^3 1(x \geq x_k).$$

This is easy enough to implement plug into `R`, but you won't have to.
It has splines integrated into its formula language.

Boundary Issues

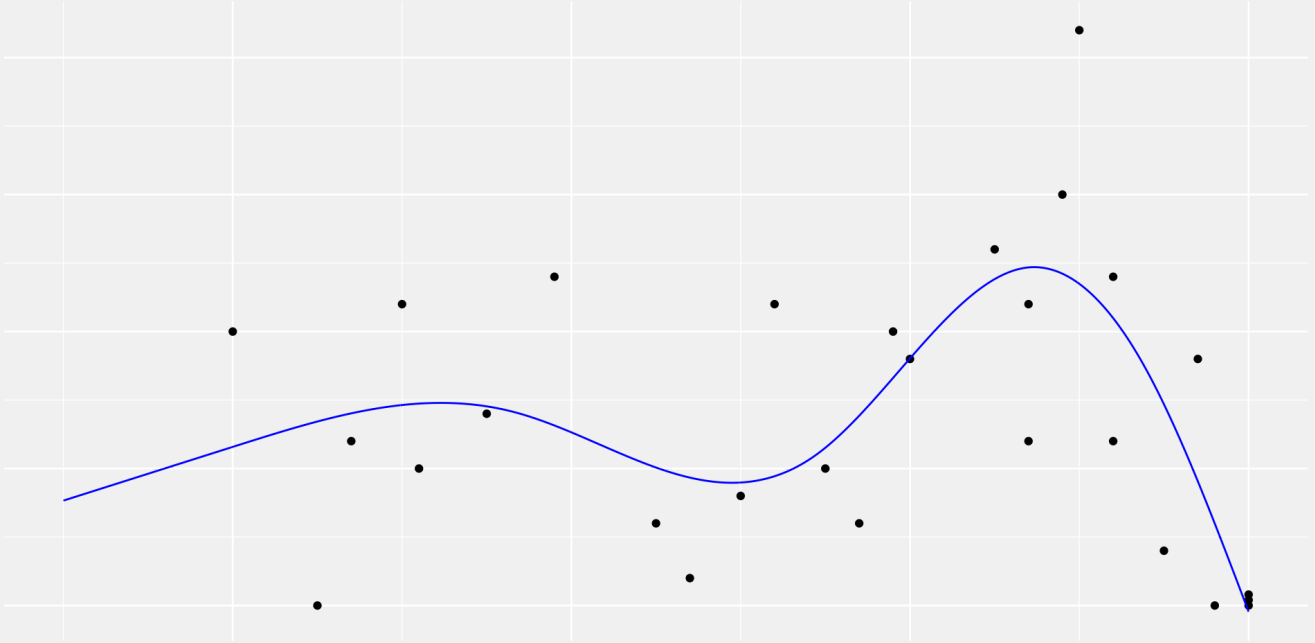
Splines like this don’t do a great job past the boundary of the data.



- Obviously we don’t know much about what we should predict there.
- But continuing a polynomial that fit in the range of the data leads to counterintuitive predictions.
- There’s no evidence here justifying the steep dive we see on the left.

Natural Splines

Natural cubic splines satisfy an additional constraint. They are linear at, and beyond, the edges of the data.



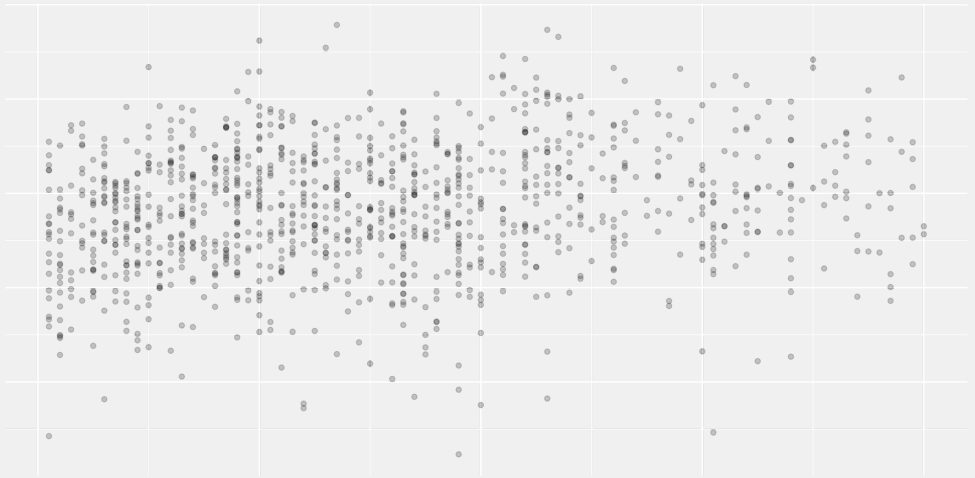
- This is probably what you want.
- Finding a basis for these is a bit more complicated.
- But `R` will do it for you.

Defaults

- Natural cubic splines are a good model to use by default.
- Next week we'll talk about a **local polynomial regression**.
- If you're doing regression in 1D, you're pretty safe using either.
- The week after next we'll talk about higher-dimensional data.
- That's where model choice gets complicated.
  - There aren't really any good universal defaults.
  - That's where you come in.

Regression with Transformed Outcomes

Sometimes the outcome we observe isn’t what we want to predict



- Maybe we’re not interested in predicting test scores themselves.
- We might be interested in predicting whether they exceed a threshold T.
- We can fit a curve that does that.

Predicting an indicator

We can and often do fit models to indicators via least squares.

$$\hat{\mu} = \underset{m \in \mathcal{M}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \{ Y_i - m(X_i) \}^2.$$

- The curve this estimates is, as usual, essentially the mean of y at a given x.
- And because y is an indicator, its mean is a **frequency**.
- It's the proportion of observations at x you expect to exceed the threhold.

Predicting an indicator

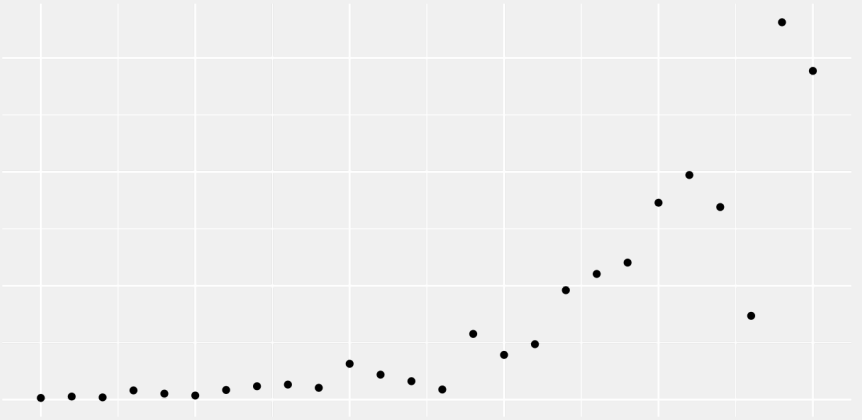
We can and often do fit models to indicators via least squares.

$$\hat{\mu} = \underset{m \in \mathrm{M}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \{Y_i - m(X_i)\}^2.$$

- The main difference is that the plots are hard to look at.
- The transformed observations all lie on top of each other.

Sometimes the outcome we observe is hard to predict linearly

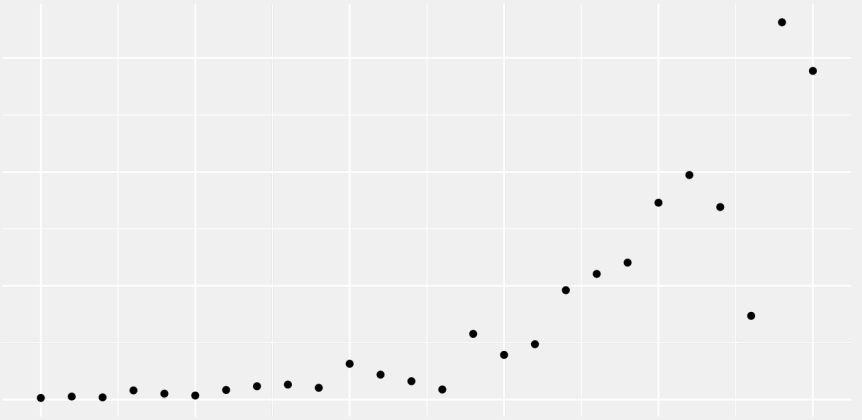
This happens, in particular, when there's **exponential growth**.



- In financial data, this arises from compound interest.
- In disease data, this happens because sick people make people sick.

Sometimes the outcome we observe is hard to predict linearly

This happens, in particular, when there's **exponential growth**.



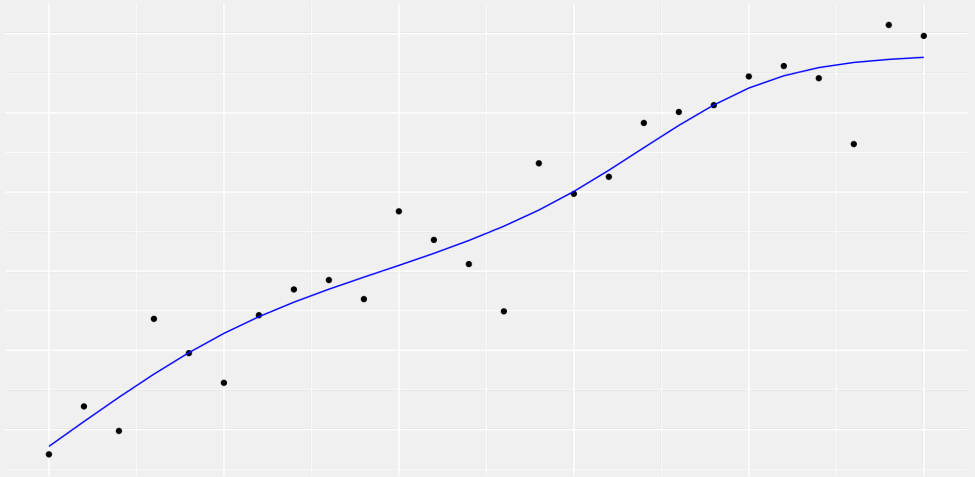
We could try to fit this outcome using a linear model for fast-growing curves.

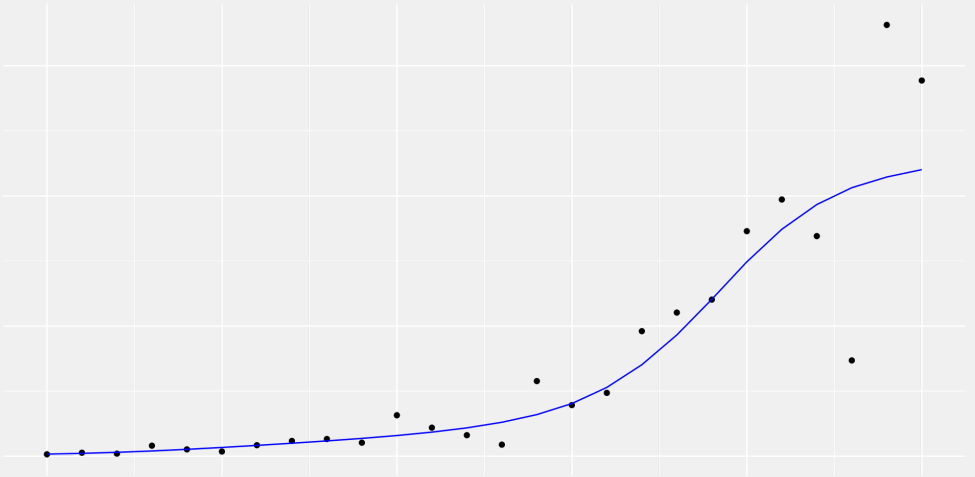
$$m_\beta(x) = \beta_0 + \beta_1 e^x + \beta_2 e^{2x} + \ldots$$

But it often makes more sense to fit the outcome's logarithm.

$$Y_i = \log(\text{outcome}_i).$$

Predicting a log





- We fit the logarithm via least squares using a more typical linear model.
- Then we exponentiate its predictions to predict the observed outcome.

Other transformations

You can fit other transformations of the outcome you observe.

$$\hat{\mu}(X_i) \approx Y_i^2 \qquad \text{squares}$$
$$\hat{\mu}(X_i) \approx \sqrt{Y_i} \qquad \text{square roots}$$
$$\hat{\mu}(X_i) \approx f(Y_i) \qquad \text{any function of the outcome at all}$$

It's very normal to do this.

- There's nothing special about the variables you have in your dataset.
- If you observe some variables, you observe all functions of them, too.
- In fact, what you observe is often not what was originally collected.
  - People transform, save, and redistribute data all the time.
  - What you get reflects the thinking of the person you got the data from.

Where Errors Come From

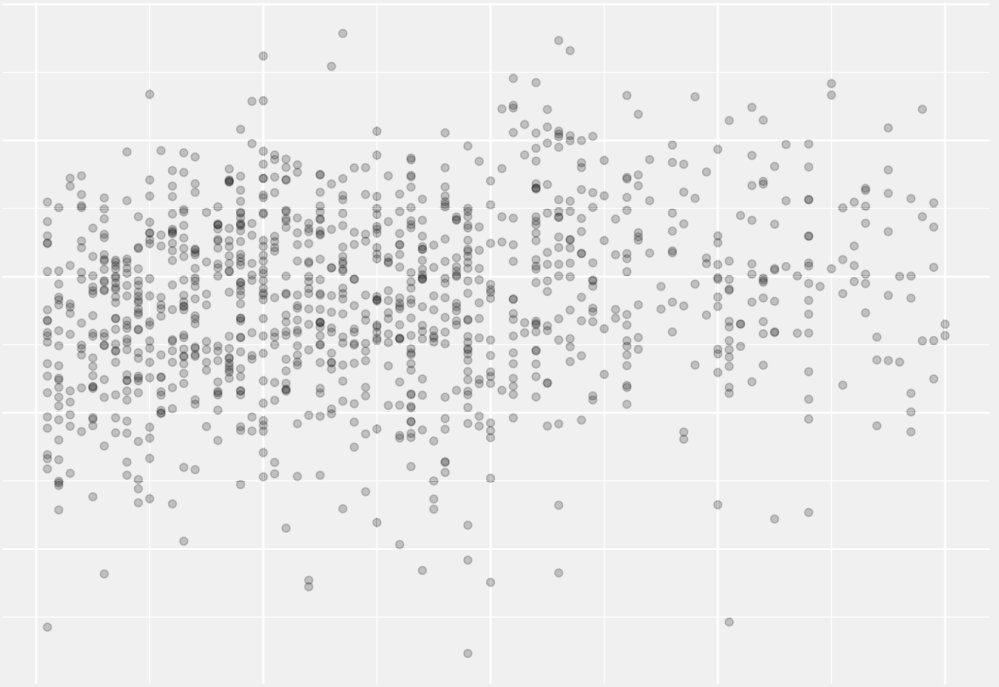
Errors in Signal Processing

In signal processing, it’s easy to think of where errors come from.

- Compression, like recoving a signal from an MP3.
- Energy leakage in digital photography.
- Quantum fluctuations in low-light photography.

In short, it’s physics stuff.
That’s why it’s nice to talk about signal processing.

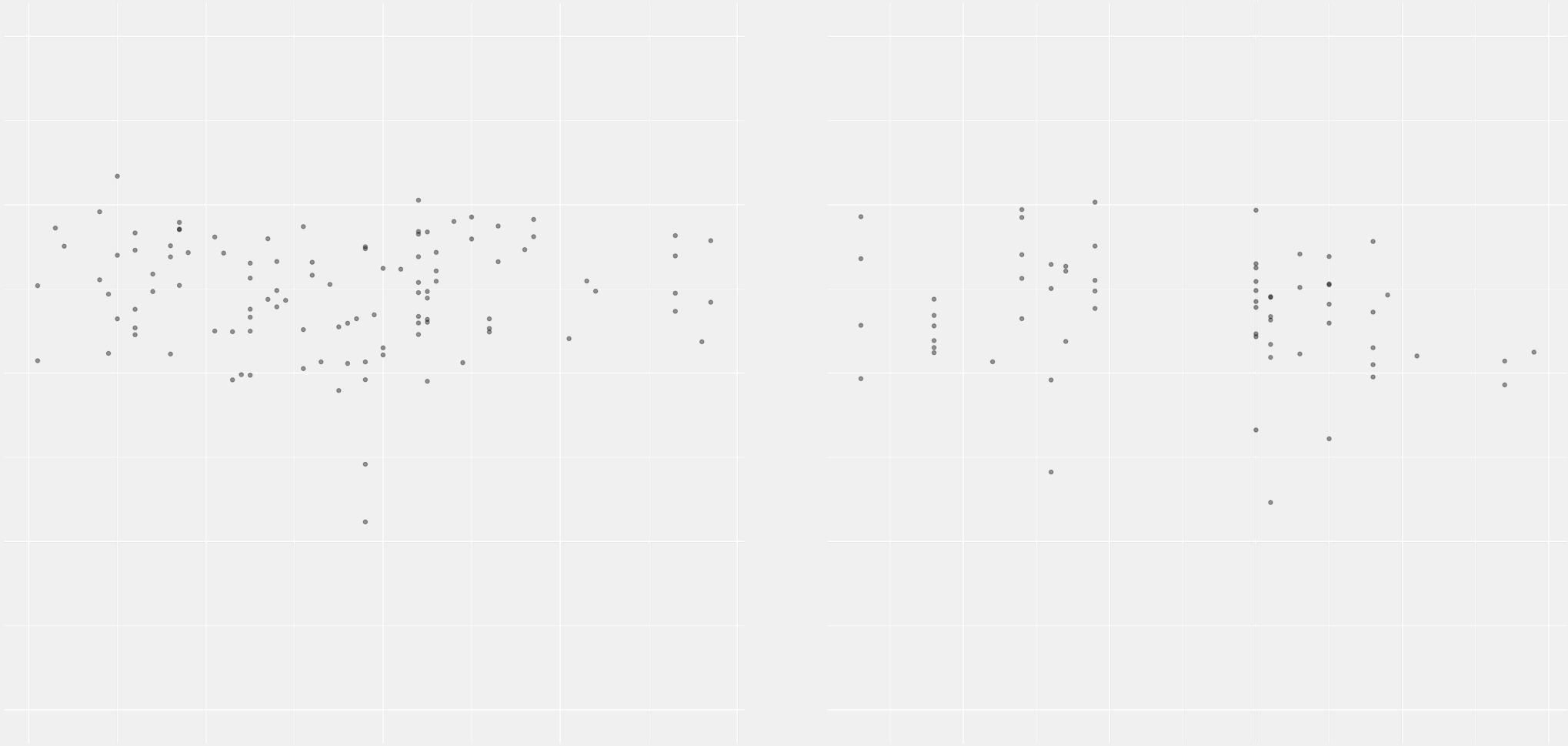
Errors in Social Science Data



In social science data, it’s not so easy.

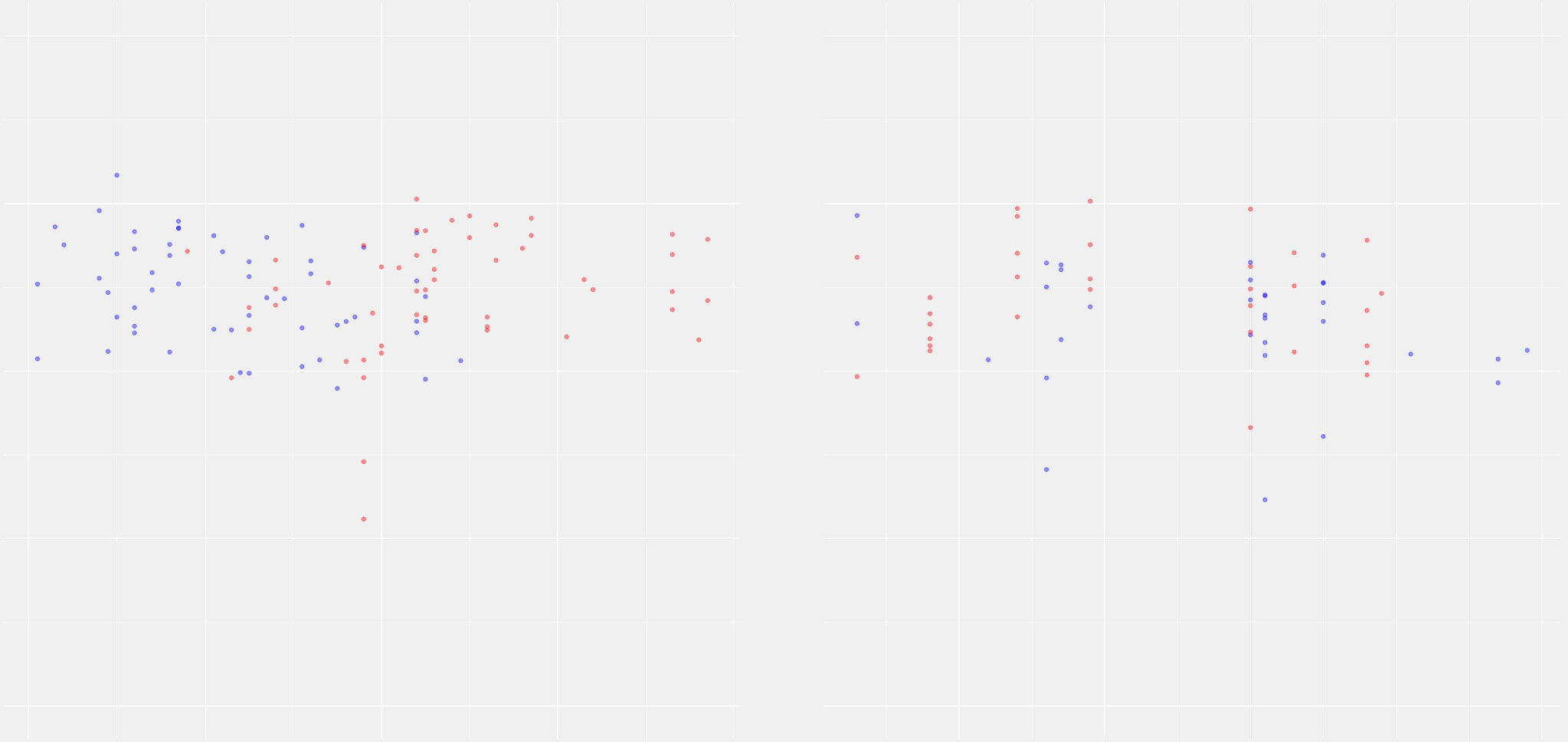
- There’s always going to be variation.
- No two people, schools, etc. are going to be exactly alike.
- But usually some of the variation we see can be explained.

Explaining Variation



If we break our test scores data down by city, some of the variation goes away.

Explaining More Variation



If we break it down by school-district population density too, even more does.

Explaining it all?

$$Y_i = \underbrace{\mu(X_i, \text{etc}_i)}_{\text{trend}} + \underbrace{\varepsilon_i}_{\text{error}}$$

Good news!

- Until we've got that language, we'll basically stick to signal processing.
- It may sound like we're talking about social science, but we'll be using fake data.
- That means we'll know exactly what the **clean signal** is and how it's compressed.
- Working with well-thought-out fake data is a good way to check your approach.
- If your approach fails on fake data, why would you trust it on the real stuff?

This Friday

- It’ll be a lab like last Wednesday’s.
- Very much like last Wednesday’s.
  - We’ll still be doing signal processing in 1D.
  - And we’ll use the same signals and everything.
- But there’ll be a few new ingredients.
  - Bigger errors.
  - Unevenly-spaced sampling.
  - Spline models.
- To prepare, make sure you have code from last week’s lab to build on.
  - You can try to get your own code into shape.
  - Or you can get comfortable with the code from the solution.
  - Or mix and match.